Resource-Constrained Neural Monte Carlo Tree Search for Quantum Boolean Oracle Synthesis

Zixiang Zhou

January 1, 2026

Abstract

Boolean oracle synthesis embeds classical predicates into quantum algorithms, but direct algebraic-normal-form constructions can be costly in non-Clifford gates, CNOTs, logical depth, and temporary workspace. We formulate bit-flip Boolean oracle synthesis as a resource-constrained search problem over ANF/FPRM term sets and introduce Resource-NMCTS, a logical-layer workflow that combines neural action priors, Monte Carlo tree search, Boolean-ring actions, frontier controllers, Pareto archives, and baseline-preserving guards. Unlike SSHR, the method does not search parallelotope candidates; SSHR is used only as a CNOT-oriented small-function baseline. On 177 traditional functions with $n \leq 6$, Pareto-Resource-NMCTS reduces mean T-count by 72.25% and mean weighted score by 67.80% relative to direct ANF synthesis, and its score win/loss/tie counts against ESOP beam, weighted ESOP-MILP, and SSHR-H are 174/0/3, 167/3/7, and 173/4/0. External probes show corresponding score wins against ROS-style LUT (309/0/0), mockturtle (166/11/0 and 64/0/0), CirKit (177/0/0 and 64/0/0), and legacy RevKit CLI exact-oracle synthesis (173/0/4). These comparisons also expose the expected tradeoffs: CirKit is often shallower, and RevKit uses fewer auxiliary lines on most small functions. In the phase branch, a rank-trained shortlist scores 512 of 8192 affine forms per held-out $n = 6$ function and matches the wide-search mean within 0.01% under the $T/R_z = 30$ proxy. For larger term-set instances, a sparse depth-2/4 frontier matches the measured depth-2/3/4 frontier while reducing evaluation time by 24.94–30.28%, and a learned depth-4 gate preserves sparse-frontier score on a 144-pair multi-seed $n = 24, 28, 32, 40$ audit while saving 13.43%. Correctness is supported by 15,774 symbolic, phase-search, and policy-pruning rows, plus 700/700 complete truth table-bridge rows for $n = 21$–30. The contribution is a verified logical-layer search method with strong T-count and weighted-score advantages, not a hardware-mapped, depth-only, or universal-dominance claim.

1 Introduction

Given a Boolean function $f : \{0,1\}^n \to \{0,1\}$, a bit-flip oracle implements

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \tag{1}$$

Such oracles appear in Grover search, constraint solving, and quantum cryptanalysis. Their cost is not determined only by the number of Boolean operations. In a fault-tolerant or resource-limited setting, the chosen Boolean representation, compute/uncompute order, temporary workspace, and phase realization all affect T-count, CNOT count, logical depth, gate count, and peak ancilla.

Existing synthesis routes provide strong local solutions. ESOP, XAG, LUT and BDD representations reduce different Boolean resources; ROS makes LUT-based oracle synthesis resource-aware; RevKit, mockturtle, Caterpillar, CirKit, and ABC provide mature logic and reversible-synthesis toolchains; and SSHR uses parallelotope structures in the Boolean hypercube to target small-scale CNOT-oriented synthesis [10, 9, 8, 24, 1, 18, 17, 26, 16, 15, 7, 14].

These methods motivate the baselines used in this paper, but they also expose a gap: a fixed representation or a single resource objective may miss useful tradeoffs when T-count, CNOTs, depth, gates, and ancilla are optimized together.

We address this gap by treating Boolean oracle synthesis as an explicit search problem. The state is an ANF/FPRM term set; actions are algebraic rewrites that can be expanded back to GF(2) semantics; neural policies and MCTS allocate the search budget; and guard rules ensure that learned candidates do not replace a stronger deterministic baseline. The resulting circuit is checked at the plan level, at the emitted X/CNOT/MCT symbolic level, and, for bridge instances, by full truth table oracle simulation.

The paper makes four contributions. First, it defines Resource-NMCTS as a resource-constrained ANF/FPRM search pipeline rather than as an SSHR extension. Second, it combines Boolean-ring linear-factor screening, neural action priors, MCTS, depth-frontier policies, Pareto archives, and baseline-preserving guards into one logical-layer synthesis workflow. Third, it evaluates the method against ESOP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, Caterpillar API, CirKit, RevKit CLI, and RevKit phase/Rz baselines on matched functions. Fourth, it separates algorithmic gains from engineering guards through ablations and high-dimensional verification bridges.

Scope and assumptions. The study is intentionally limited to logical-layer Boolean-oracle synthesis. All reported resources are computed before hardware mapping, routing, native-gate scheduling, noise modeling, and magic-state-factory accounting. The weighted score in Eq. (3) is used only to rank verified candidates; T-count, CNOT count, depth, gate count, and peak ancilla are also reported separately, and cross-toolchain comparisons are interpreted through the baseline comparability audit rather than as a universal leaderboard.

Table 1: Contribution-to-evidence map. Each contribution is tied to concrete implementation mechanisms, paper evidence, and the boundary that prevents the claim from being overstated.

Contribution	Implementation	Paper evidence	Boundary
Resource-constrained Boolean-oracle search formulation	ANF/FPRM term-set state, Boolean-ring and affine actions, guarded candidate selection, symbolic circuit emission.	Fig. 1; Table 3	Logical-layer oracle synthesis only; no routing, native-gate mapping, or noise model.
Neural/MCTS resource-search workflow	Neural action priors, MCTS/beam/portfolio search, Pareto archive, frontier policies, staged and sparse learned gates.	Tables 37, 38, 49; Fig. 5	The largest gains come from searchable algebraic actions plus guarded selection; the paper does not claim deep RL alone explains all gains.
Broad baseline and toolchain comparison	Direct ANF, ESOP beam/MILP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, Caterpillar API, CirKit, RevKit CLI, and phase/Rz probes.	Tables 8, 9, 33; Fig. 3	Strong T-count and weighted-score evidence, not universal CNOT/depth/ancilla dominance or full ROS reproduction.
High-dimensional and phase-search verification envelope	$n = 20, 24, 28, 32, 40$ symbolic term-set checks, $n = 48, 56, 64$ ultra-scale symbolic stress, $n = 21$–30 complete truth-table bridges, Affine-FPRM phase search, learned phase shortlist.	Tables 61, 62, 63, 64, 60; Fig. 7	Large rows are logically and symbolically verified; complete truth-table enumeration remains limited to bridge slices.

Table 2 then rewrites the same positioning as reviewer-facing questions. Its purpose is not to add a new metric, but to show why the comparison set is meaningful while preserving the limitations that make each comparison fair.

Table 2: Novelty and comparison scorecard. Each row answers a reviewer-facing comparison or novelty concern, points to the manuscript evidence, and states the boundary that prevents overclaiming.

Reviewer question	Answer	Evidence	Boundary
Is this only an SSHR variant?	No. Resource-NMCTS searches ANF/FPRM and Boolean-ring actions; SSHR is a CNOT-oriented small-function counterpoint.	Tables 1, 16	SSHR remains a strong CNOT reference; the paper claims T-count and weighted-score gains, not CNOT dominance.
Are the primary baselines matched to the oracle task?	Yes. Direct ANF, AND-direct ANF, ESOP/MILP, BDD/ABC, and SSHR rows are paired on the same Boolean-oracle functions and resource score.	Tables 9, 33	These prove matched logical-resource improvements, not hardware-mapped optimality.
Does the comparison go beyond internal baselines?	Yes. The package includes ROS-style LUT, mockturtle, Caterpillar API, CirKit, RevKit CLI, ABC/BDD, and RevKit phase/Rz probes.	Fig. 3; Tables 23, 28, 32	External rows are logic-level or exact-oracle probes; they are not full hardware mapping or full ROS reproduction.
Does a score win hide unfavorable resources?	No. Counterpoint, Pareto, and schedule-proxy audits report where SSHR, Caterpillar, CirKit, and RevKit remain strong.	Tables 16, 36, 48	The correct claim is a strong T/weighted-score point with explicit CNOT, depth, ancilla, and runtime tradeoffs.
Is AI/MCTS actually isolated from deterministic search?	Yes. The search-control audit separates heuristic, beam, no-MCTS, MCTS, Pareto, learned-prior, random-prior, random-depth, and phase controls.	Tables 38, 49	The learned components are promoted only where they give bounded quality, pruning, or budget-allocation evidence.
Does the study go beyond small truth-table functions?	Yes. Large $n = 20$, 24, 28, 32, 40 symbolic term-set runs and an $n = 48$, 56, 64 ultra-scale stress slice are paired with $n = 21$–30 complete truth-table bridge slices.	Tables 61, 62, 63; Fig. 7	Large-dimensional evidence is symbolic plus bridge-verified, not exhaustive truth-table benchmarking for all large n.

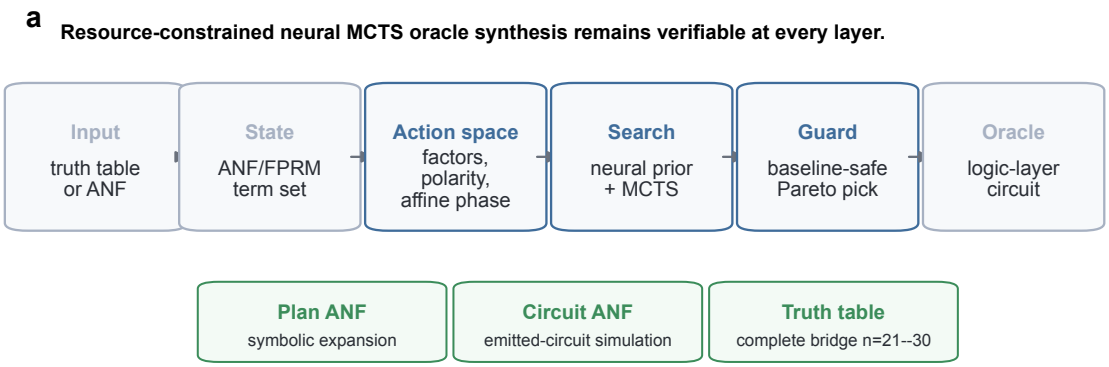

Figure 1: **Verified search pipeline.** Resource-NMCTS operates on complete truth table inputs or ANF term sets, searches algebraic actions under neural and tree-search control, and verifies candidate circuits at plan, emitted symbolic-circuit, and complete-truth table levels.

2 Related Work

2.1 Boolean representations for oracle synthesis

Low-T oracle synthesis is closely tied to the Boolean intermediate representation. BDD-based reversible synthesis uses symbolic decision diagrams to handle large functions [24]. ROS frames oracle synthesis as resource-constrained LUT mapping with downstream reversible implementation costs [10]. XAG-based compilation separates XOR structure from AND nodes,

making multiplicative complexity a proxy for non-Clifford cost [9, 8]. Back-end-aware oracle synthesis further connects logic synthesis to downstream resource metrics, but that line includes mapping concerns that are outside the scope of this logical-layer study [25].

2.2 Reversible and logic-synthesis toolchains

ABC supplies mature AIG, XAG/GIA, LUT, and ESOP optimization flows for classical logic [1]. mockturtle, Caterpillar, and CirKit provide reusable logic-network optimization libraries and shells in the EPFL logic-synthesis ecosystem [18, 15, 7, 14]. RevKit provides reversible-logic and quantum-compilation functionality, including the Python `oracle_synth` API and legacy command-line reversible synthesis flows [17, 16]. In this work these toolchains are used as external baselines or probes. They are not called inside Resource-NMCTS, which keeps the proposed search policy separated from the comparison backends.

2.3 Learning-guided synthesis

Learning-guided synthesis has been explored for both classical and quantum circuits. ShortCircuit uses AlphaZero-style search for classical Boolean circuit design [20]. Nested MCTS has been used for automated ansatz and circuit design [22]; QSeed and diffusion models learn unitary or circuit generators [23, 3]; Gumbel AlphaZero and dynamic-circuit AlphaZero target Clifford+T or discrete unitary synthesis [11, 21]; and recent reinforcement-learning or supervised-search systems optimize non-Clifford counts, Pauli-network blocks, ZX rewrites, and residual unitary synthesis [4, 2, 12, 19]. MCTS has also been used for quantum circuit transformation under connectivity constraints [27], and MonteQ uses MCTS for Hamiltonian-simulation circuit synthesis [5]. The present work is different in scope: it is a logical-layer Boolean-oracle synthesizer, not a hardware-routing or transpilation method, not a Hamiltonian-simulation scheduler, not a gate-by-gate unitary generator, not a generic learned unitary compiler, and not an SSHR parallelotope search. Its learned component ranks verifiable Boolean-algebraic actions, while ESOP, BDD, XAG/LUT, ROS-style, SSHR, RevKit, mockturtle, Caterpillar, CirKit, and ABC flows define the comparison set.

Table 3 summarizes this positioning. It is used to keep the manuscript's contribution narrow and testable: existing tools and representations define strong baselines or motivation, while Resource-NMCTS is claimed only as a logical-layer resource-aware search method over verifiable Boolean-algebraic actions.

Table 3: Related-work positioning matrix. The table states what each literature family optimizes, which gap motivates the proposed search, and how the manuscript uses that family in experiments or claim boundaries.

Line of work	Representative work	Main lever	Gap for this paper	Manuscript use
BDD-based reversible synthesis	BDD reversible synthesis [24]	Symbolic decision-diagram representation for large Boolean functions.	Representation scalability does not by itself optimize T, CNOT, depth, gates, and ancilla jointly.	Used as a representation-level baseline family, not as the proposed search space.
LUT/ROS-style oracle synthesis	ROS, STG benchmarks, and back-end-aware oracle synthesis [10, 6, 25]	Resource-aware LUT mapping, small-function optimum libraries, and downstream implementation-cost estimates.	Full ROS garbage-management and hardware/back-end mapping are outside the logical-layer claim, and STG is a small-function optimum counterpoint.	Used through a verified ROS-style LUT proxy, line-aware reselection stress tests, and a published STG counterpoint.
XAG and multiplicative complexity	XAG compilation and multiplicative-complexity oracle cost [8, 9]	Separates XOR-linear structure from AND nodes as a non-Clifford proxy.	A fixed network optimization flow may miss algebraic FPRM/ANF search actions and Pareto tradeoffs.	Used through ABC, mockturtle, and CirKit probes plus exact small XAG lower-bound checks.
Logic and reversible toolchains	ABC, EPFL libraries, RevKit, mockturtle, Caterpillar, and CirKit [1, 18, 17, 16, 15, 7, 14]	Mature optimization passes and reversible-synthesis flows.	Tool output defines strong baselines but does not expose the neural/MCTS algebraic action policy.	Used as external probes, API-level implementation-family checks, and exact-oracle reversible-synthesis comparisons.
SSHR geometric synthesis	Parallelotope-based CNOT-oriented synthesis [26]	Small-scale Boolean hypercube geometry targeting CNOT-oriented covers.	The objective is CNOT-oriented and small-function focused, not a general resource-weighted AI search.	Used as a CNOT-oriented small-function baseline; not used by the proposed method.
Learning-guided circuit synthesis	AlphaZero, diffusion, reinforcement learning, supervised search, MCTS, and ZX-guided synthesis [20, 22, 23, 3, 11, 21, 4, 2, 12, 19, 27, 5]	Learns or searches over circuit-design, unitary-synthesis, block-resynthesis, and circuit-transformation actions.	Most targets are classical circuits, ansatz design, unitary synthesis, transpilation/block resynthesis, ZX rewriting, or Hamiltonian simulation rather than Boolean-oracle ANF/FPRM term search.	Motivates neural/MCTS control while keeping actions verifiable at the Boolean-algebraic oracle layer.

3 Problem and Resource Model

We represent a Boolean function by its square-free ANF

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \tag{2}$$

where each monomial m is stored as a bit mask. A direct construction applies one controlled action per monomial. The synthesis problem is to replace this direct construction with reusable compute/action/uncompute subplans that implement Eq. (1) while lowering a multi-resource objective.

All resource numbers in this paper are logical-layer estimates over X, CNOT and multi-controlled Toffoli gates. Candidate selection uses

$$\text{score} = T + 0.04\,\text{CNOT} + 0.015\,\text{depth} + 0.01\,\text{gates} + 2\,\text{ancilla}. \tag{3}$$

The score is a ranking objective, not a physical runtime or error model. We therefore report T-count, CNOT count, depth, gate count, and peak ancilla separately whenever a tradeoff is relevant.

4 Method

4.1 Search state and actions

The Resource-NMCTS state is the current set of unsynthesized ANF terms. An action chooses a reusable algebraic structure, synthesizes that structure into a temporary line, uses it to control a quotient subproblem, and then uncomputes it. Candidate actions include common monomial factors, FPRM polarity rewrites, CNOT-computed linear parity factors, Boolean-ring linear factors, and bounded affine changes of variables. Each action is scored by its immediate resource estimate and by the size and structure of the induced subproblems.

Table 4 summarizes the complete execution path. The central invariant is that learning and tree search only control which verifiable algebraic candidates are evaluated or selected; semantic correctness is supplied by the ANF, emitted-circuit, truth-table, or phase verifier attached to the corresponding stage.

Table 4: End-to-end synthesis and verification workflow. Each stage records the mechanism used by Resource-NMCTS, the invariant that prevents the learned search from becoming a semantic oracle, and the artifact produced for downstream analysis.

Stage	Mechanism	Invariant	Artifact
Input normalization	Convert a truth table, exported benchmark, or supplied term set into a square-free ANF monomial mask set $\mathcal{M}$.	The term set must evaluate to the target Boolean function before any search step is trusted.	ANF terms, preset name, and source manifest.
Candidate generation	Generate direct ANF, FPRM polarity, common-factor, Boolean-ring linear-factor, affine, and depth-frontier candidates.	Every action must have an explicit GF(2) expansion back to the original variables.	Candidate logical plans with resource estimates.
Search control	Use neural action priors, MCTS/beam expansion, staged frontiers, and sparse learned gates to allocate the search budget.	Learned decisions rank or skip evaluations; they do not define semantic correctness.	Scored candidate pool and search-time diagnostics.
Guarded selection	Compare learned and expensive candidates with deterministic baselines, then apply Pareto filtering before active-score selection.	A selected plan must be a valid generated candidate and remains bounded by the stated guard and score model.	Selected logical plan and non-dominated archive entry.
Circuit emission	Expand the selected plan into X/CNOT/MCT compute-action-uncompute operations and compute logical resource counts.	The reported score is a ranking objective; T, CNOT, depth, gates, and ancilla are retained separately.	Logical circuit proxy and resource row.
Verification and reporting	Check the plan algebraically, check emitted-circuit ANF semantics, add full truth-table or phase checks where feasible, and write raw CSV plus manifest rows.	Paper comparisons use matched rows that pass the relevant semantic checks and exclude errors, skips, and timeouts.	`raw_*.csv`, `summary_*.csv`, `manifest_*.json`, and paper tables.

Table 5 gives the algorithmic contract used by the implementation. It is written as a source-anchored contract rather than as language-only pseudocode: each row identifies the implementation anchors that make the corresponding method statement executable and auditable. This is important for the learning components, because the neural scorer and learned frontier gates are only allowed to change ranking, budget allocation, or skipping decisions; correctness and final resource reporting remain attached to explicit algebraic expansions, guarded candidate selection, and semantic verification.

Table 5: Algorithm contract for Resource-NMCTS. The table converts the method description into implementation-anchored operational rules and reviewer-facing guarantees.

Stage	Operational rule	Source anchors	Reviewer-facing guarantee
Canonical state	Normalize a truth table, exported benchmark, or supplied term list into a square-free ANF monomial set; direct ANF remains the fallback plan at every state.	synthesizers.py: anf_monomials; factor_plan.py: frozenset, direct_plan	The search starts from the same Boolean function as the baseline and always has a syntactically valid logical oracle construction.
Action generation	Generate monomial factors, linear/Boolean-ring factors, FPRM polarity actions, and affine/linear-pair candidates only when they have an explicit GF(2) expansion.	factor_plan.py: candidate_actions, linear_factor_actions, _linear_boolean_expansion; synthesizers.py: affine_linear_pair_beam_plan	Learning never invents a semantic rule; it can only rank or select among algebraic actions that the emitter and verifier can expand.
Prior-guided tree search	Use the neural scorer to bias action priors, then run PUCT/MCTS recursion with greedy rollouts and bounded candidate_top_k expansion.	neural_policy.py: NeuralScorer, score_many; nmcts_solver.py: NeuralMCTSSolver, c_puct; factor_plan.py: candidate_top_k	The neural component is a budget-allocation prior, not a correctness oracle or an unrestricted circuit generator.
Baseline guard	Evaluate deterministic, beam, MCTS, neural, and screen candidates as a portfolio; guarded neural variants return the better of the baseline and learned candidate.	synthesizers.py: _run_child_portfolio, min([baseline, neural_plan], _portfolio_result, direct_anf	A learned or expensive branch is allowed to help but is not allowed to replace a stronger verified deterministic candidate in guarded rows.
Pareto archive	Collect candidates from multiple resource profiles, remove candidates dominated in T, CNOT, depth, gates, and peak ancilla, then select by the active score.	synthesizers.py: _pareto_front, _dominates, _resource_selection_key, pareto_resource_nmcts	Weighted-score wins are kept separate from multi-resource dominance, which is why tradeoff tables remain part of the Results section.
Large-scale controllers	Use depth-frontier policies and sparse gates to decide which high-dimensional Boolean-ring screens to evaluate, with deterministic sparse/full frontiers kept as references.	run_truth_bridge_terms.py: load_depth2_guard, load_frontier_policy, depth_frontier_policy, screen_depth4	High-dimensional controllers are planning-time controls over symbolic term-set verification, not hardware schedulers or exhaustive truth-table solvers.
Emission and verification	Emit X/CNOT/MCT compute-action-uncompute circuits, verify ANF semantics symbolically, and use complete truth-table or phase checks where feasible before writing raw result rows.	factor_plan.py: emit_plan_to_circuit, verify_plan_anf, verify_circuit_anf, verify_oracle; run_experiments.py: correct	Paper comparisons consume matched rows that passed the relevant semantic check; skipped, errored, or timed-out rows are not silently counted.

Table 6 records the corresponding search-budget contract. The table is intentionally operational: it states the configured candidate fan-out, auxiliary-line, MCTS, polarity-search, portfolio, Pareto, frontier-controller, and verification caps that make the method rerunnable. These caps are part of the claim boundary. They explain why the method scales as a bounded logical-layer search workflow while also making clear that the reported candidates are not certificates of global optimality.

Table 6: Search-budget contract for Resource-NMCTS. Each row states the explicit cap used to make the search rerunnable, the scalability role of that cap, and the limitation it leaves in place.

Budget family	Explicit cap	Scalability role	Boundary
Action fan-out	candidate_top_k=24; max_factor_size=5; min_factor_count=2	Bounds the number of algebraic actions expanded at each ANF state.	The cap controls search cost; it is not a proof that the globally best factor action is always included.
Ancilla and recursion	max_factor_ancilla=4; depth guard=64 in MCTS recursion	Prevents recursive factoring from allocating unbounded temporary factor lines.	This is a resource guard, not an ancilla-optimality theorem.
PUCT/MCTS simulations	mcts_simulations=96; neural_mcts_simulations=128; c_puct=1.35	Makes tree-search compute an explicit, rerunnable budget rather than an open-ended optimizer.	The MCTS budget supports a bounded search-control claim, not exact optimal synthesis.
Polarity and affine search	max_polarities=384; high-dimensional direct screening narrows expensive planning trials	Keeps fixed-polarity and affine preconditioning searches tractable on larger sparse term sets.	Screening is a controlled heuristic; missed polarities remain part of the search-boundary limitation.
Resource portfolio	fast_config caps candidate_top_k$\leq$18, MCTS$\leq$32/40, max_polarities$\leq$16/24 before portfolio selection	Separates cheap deterministic candidates from more expensive MCTS or affine branches.	Portfolio selection gives a best verified candidate among attempted children, not a certificate over all possible children.
Pareto archive	Deduplicate by (T,CNOT,depth,gates,peak ancilla), remove dominated candidates, then rank the non-dominated front by active score	Keeps multi-objective diversity without carrying every generated candidate into the final comparison.	The archive is Pareto over generated candidates only; it is not a global Pareto frontier.
High-dimensional frontier controllers	Staged, sparse, and learned gates decide whether depth-3/depth-4 Boolean-ring screens are evaluated	Moves large-term-set runs from all-frontier evaluation toward validated budget control.	The controller is empirical on the audited slices and is not a proof that depth-4 can always be skipped.
Verification route	Symbolic ANF/circuit checks for large rows; complete truth-table bridge slices where enumeration is feasible	Allows high-dimensional evidence without pretending that every large instance was exhaustively enumerated.	Symbolic verification is exact for emitted GF(2) semantics but does not replace hardware mapping or exhaustive enumeration for all n.

4.2 Boolean-ring linear factors

A key extension is allowing the linear factor and the quotient to share variables under Boolean-ring semantics. For example,

$$(x_i \oplus x_j)x_i m = x_i m \oplus x_i x_j m, \tag{4}$$

because $x_i^2 = x_i$ over the Boolean ring. Circuit-wise, the parity $x_i \oplus x_j$ is computed by CNOTs into a temporary line, used as a control for the quotient plan, and uncomputed. This expands the algebraic action space without changing the GF(2) verifier.

4.3 Neural prior, MCTS, frontier policy, and guard

The neural action prior ranks candidate actions using term-count, degree, coverage, variable-support, and immediate-resource features. MCTS then explores combinations of actions with deterministic rollout estimates. A separate frontier policy selects among depth-2, depth-3, and depth-4 Boolean screening frontiers; a cost-aware version trades a small score increase for lower planning time and reduced ancillary lifetime area. We also use two conditional frontier controllers. The staged controller evaluates depth-2 and depth-3 screens first and only runs depth-4 when the measured depth-3 improvement is large enough. The sparse depth-4 gate instead uses the depth-2 state to predict whether the deterministic sparse depth-2/4 frontier needs its depth-4 evaluation; its threshold is selected on validation data to avoid false skips of improving depth-4 cases. Finally, a guard compares learned or expensive candidates to deterministic baselines and

returns the best valid candidate under Eq. (3). This makes learning a search-control component, not the source of semantic correctness.

4.4 Pareto archive

The Pareto variant evaluates candidates under several internal objectives, including active-score, T-sparse, CNOT/depth-heavy, gate-sensitive, and ancilla-tight rankings. It removes candidates dominated simultaneously in T-count, CNOT count, depth, gate count, and peak ancilla, then selects from the remaining archive using the active score. The archive is used only after each candidate has passed the same plan and emitted-circuit checks as the base method.

4.5 Phase/Rz branch

RevKit `oracle_synth` returns phase/Rz netlists rather than direct bit-flip Clifford+T circuits. To avoid treating that boundary only as a cost sensitivity, we also implement a phase-parity emitter: each ANF monomial is expanded into merged parity-phase gadgets and verified up to global phase. Fixed-polarity FPRM search and bounded affine preconditioning are then added as phase-search actions. This branch is reported as a logical phase/Rz proxy; it is not a final Clifford+T compiler or mapped hardware circuit, although a separate coarse sequence-smoke audit checks representative single-qubit rotations.

5 Experimental Design

Experiments answer four questions. First, does Resource-NMCTS reduce T-count and weighted score on small functions where full truth table checking is practical? Second, do the conclusions survive comparisons with ESOP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, Caterpillar API, CirKit, RevKit CLI, and RevKit phase/Rz baselines? Third, which components–affine search, MCTS, neural priors, guards, frontier policies, and Pareto archives–account for the gains? Fourth, how far can correctness evidence be pushed beyond small truth tables?

The traditional slice contains 177 functions with $n \leq 6$. High-dimensional logic-network probes include exported $n = 14$, $n = 15$, $n = 16$, and $n = 18$ random-ANF sets for ABC/BDD, and $n = 14$ sets for mockturtle and CirKit. Scale and bridge experiments cover item-level $n = 20, 24, 28, 32, 40$ term sets, an $n = 48, 56, 64$ ultra-scale symbolic stress slice, and complete truth table bridge functions at $n = 21$–30. Paired comparisons include only matching functions or items with successful semantic checks and no timeout. Table 7 summarizes this benchmark-suite composition before any leaderboard-style result is reported. It separates enumerable small Boolean oracles, published tiny-function optima, external logic-network probes, reversible/phase probes, symbolic large-n stress tests, complete truth table bridges, and learned-control rows. This table is included to make the benchmark representativeness boundary auditable: the package contains broad logical-layer evidence, but no row is allowed to stand for exhaustive coverage of all Boolean functions or for hardware-mapped performance.

Table 7: Benchmark-suite composition audit. Each row reports the role, variable range, item count, raw row count, checked row count, verification route, and residual representativeness boundary for one benchmark family.

Suite	Role	Scope	Items	Rows	Checked	Verification route	Boundary
Matched small Boolean oracles	primary same-task benchmark	$n = 3$–6	177	3819	3819	complete truth table for n≤6 rows; timeout/skipped rows remain explicit	Small functions are enumerable and fair for method comparison, but they are not a large-n optimality claim.
Published tiny-function optimum counterpoint	optimum-library boundary	$n = 4$–5	54	270	270	public n=4/5 truth tables with complete correctness checks	Used as a small-function boundary, not as evidence that the proposed search beats precomputed optima.
External logic-network probes	toolchain stress test	$n = 3$–18	373	968	968	export/readback, ABC BLIF/Verilog, XAG/AIG/API checks where available	These are logical probes, not full ROS SAT garbage management, reversible emission, routing, or hardware mapping.
RevKit reversible and phase probes	reversible/phase counterpoint	$n = 3$–6	181	10286	10109	exact reversible permutation checks, phase verification up to global phase, and coarse rotation-sequence smoke checks	R_z rows are proxy-level phase evidence and do not constitute final approximate Clifford+T rotation synthesis.
Large symbolic term-set scaling	scalability and stress test	$n = 20$–64	384	7392	7392	symbolic ANF and emitted-circuit ANF checks	Symbolic large-n rows are not exhaustive truth-table enumeration and do not prove global optimality.
Complete truth-table bridge slices	large-n semantic bridge	$n = 21$–30	60	880	880	complete truth-table construction plus plan and emitted-circuit checks	The bridge is intentionally narrow because full truth tables grow exponentially.
Learned-control and stochastic controls	causal attribution and stability control	$n = 3$–40	225	16074	15930	same-budget random controls, held-out splits, independent seeds, and semantic checks	These rows support bounded control gains; they do not imply that deep learning alone causes the main resource reduction.

Because these baselines operate at different abstraction levels, Table 8 first separates their role in the argument from the claims they cannot support. Table 9 then summarizes the quantitative comparison evidence for each baseline family.

Table 8: Baseline claim matrix. The table explains why each comparison family is included, what conclusion it supports, and which stronger conclusion is outside the scope of the logical-layer experiments.

Role	Compared methods	Why it matters	Supported claim	Excluded claim
Primary resource-efficiency baselines	Direct ANF, AND-direct ANF, ESOP beam/MILP, BDD/ABC-ESOP, SSHR-H/SSHR-I	These methods solve the same bit-flip Boolean-oracle task under the paper's logical resource model.	Resource-NMCTS improves T-count and weighted score on matched small functions while SSHR remains a strong CNOT counterpoint.	Does not prove universal CNOT, depth, ancilla, or hardware-level optimality.
External logic-network probes	ABC AIG/XAG/LUT, ROS-style LUT and garbage-budget proxy, mockturtle KLUT-to-XAG, Caterpillar ANF-XAG API, CirKit AIG/MC	They test whether mature Boolean-network optimization already removes the apparent advantage over direct algebraic baselines.	The score advantage persists against stronger exported-network, official-header, and local implementation-family API probes.	Does not reproduce full ROS SAT garbage management or reversible/hardware mapping.
Exact reversible-synthesis probe	Legacy RevKit CLI TBS/DBS/RMS exact-oracle portfolio	It embeds each function as $(x, y) \mapsto (x, y \oplus f(x))$ and checks a genuine reversible-synthesis toolchain.	Resource-NMCTS has lower T-count and weighted score against this portfolio but pays more auxiliary lines.	Does not imply lower line count, routed depth, or mapped Clifford+T cost.
Phase/R_z branch	RevKit oracle_synth, phase-parity ANF, fixed-polarity FPRM, affine FPRM, learned phase shortlist	It tests whether the same search framing extends to phase-oracle and affine-preconditioned R_z cost proxies.	Affine and learned candidate pruning improve verified logical phase/R_z proxy objectives.	Not a final Clifford+T decomposition; only a coarse sequence-smoke audit emits bounded single-qubit strings.
Internal search ablations	No-MCTS portfolio, learned prior off, guard off, Pareto archive off, depth-frontier variants, sparse gate	They separate representation effects from the contribution of search, neural ranking, guards, and budget control.	AI-guided search and guards add measurable gains or time savings beyond deterministic construction.	Does not support a claim that deep reinforcement learning alone causes the whole improvement.
Scaling and correctness bridges	$n = 20$, 24, 28, 32, 40 term-set symbolic runs; $n = 48$, 56, 64 ultra-scale symbolic stress; $n = 21$–30 complete truth-table bridge rows	They push semantic checks beyond the small truth-table slice while keeping full verification where still feasible.	The emitted logical-layer circuits remain symbolically and bridge-truth-table verified at larger dimensions.	Does not make exhaustive high-dimensional truth-table benchmarking feasible or complete.
Trade-off audits	Raw multi-resource dominance, line-aware LUT reselection, schedule proxy, sparse-threshold sweep	They prevent a weighted-score result from hiding CNOT, depth, line-count, or planning-time tradeoffs.	The method occupies a strong T/score point with explicit depth, CNOT, ancilla, and runtime boundaries.	Does not justify reporting a single-metric victory as complete dominance.

Table 9: Reviewer-facing evidence matrix. The table consolidates the baseline family, coverage, verification count, headline paired result, and scope boundary used by the manuscript claims.

Evidence block	Scope	Verified rows	Main score result	Boundary
Internal logical baselines	$n = 3$–6	1770/1770	Pareto vs direct ANF 172/1/4, -67.80%; vs ESOP-MILP 167/3/7, -29.84%	Same verifier/cost model; includes direct, AND-direct, ESOP beam/MILP, SSHR-H, MCTS, affine, and Pareto variants.
Exported SSHR/ABC/BDD extension	$n = 3$–6	1416/1416	Resource-NMCTS vs SSHR-I CNOT 172/5/0, -29.48%; vs ABC-XAG 177/0/0, -64.15%	SSHR-I rows use an 8 s Gurobi budget; ABC/BDD rows are logical estimates over exported truth tables.
ROS-style LUT proxy	$n = 3$–6, 14, 15, 16, 18	309/309 best rows; 927/927 K-sweep rows; 927/927 garbage-proxy rows; 1059/1059 garbage-budget rows	Pareto vs best-K proxy 309/0/0, -84.27%	Verified LUT and executable garbage-pressure proxies only; no official ROS SAT garbage management, reversible emission, or hardware mapping.
Published STG optimum-library counterpoint	n=4-5	270/270	Pareto vs STG T-count optimum 0/40/14, +84.26%; vs direct ANF on STG slice 50/4/0, -58.10%	Public n=4/5 spectral-representative optimum-library table; this is a strong small-function counterpoint, not a reproduced ROS flow or scalable compiler baseline.
mockturtle KLUT-to-XAG probe	$n = 3$–6, 14	241/241	$n \leq 6$ 166/11/0, -31.50%; $n = 14$ 64/0/0, -91.49%	Official-header XAG resynthesis probe; still a logical proxy, not full ROS or reversible garbage management.
Caterpillar XAG API probe	$n = 3$–6	177/177 best rows; 531/531 strategy rows	Pareto vs best API score 177/0/0, -47.94%; CNOT 4/173/0, +195.18%	Local Caterpillar logic-network API over ANF-XAG inputs; verified implementation-family performance probe, not full ROS SAT garbage management.
CirKit AIG/MC probe	$n = 3$–6, 14	241/241	$n \leq 6$ 177/0/0, -62.34%; $n = 14$ 64/0/0, -94.46%	AIG multiplicative-complexity probe; strongest current depth-oriented external counterpoint.
Legacy RevKit CLI exact oracle	$n = 3$–6	708/708	Pareto vs best-score portfolio 173/0/4, -67.28%	Exact reversible oracle permutation; CNOT/depth are derived from Toffoli-control distributions and ancilla is a visible tradeoff.
RevKit phase/Rz branch	$n = 3$–6	531/531	Affine phase vs RevKit oracle_synth 177/0/0, -65.50%	Logical phase/Rz proxy verified up to global phase; sequence evidence is a coarse smoke check, not optimal Clifford+T synthesis.
Learned phase pruning	$n = 3$–6	7611/7611	Held-out n=6 rank-diverse top512 vs wide128 0/7/31, +0.00%	Policy-pruned affine candidate ranking; evidence is held-out exact scoring, not a standalone compiler backend.
High-dimensional frontier search	$n = 20$, 24, 28, 32, 40, 48, 56, 64	2088/2088	Stage-gated vs all-depth scale 0/4/92, +0.04% score; -25.43% time; ultra $n = 48$, 56, 64 stress 480/480 plan+circuit verified	Term-set symbolic verification with emitted-circuit ANF checks; depth frontier is a planning guard, not a hardware scheduler.
Complete truth-table bridges	$n = 21$–30	700/700	Bridge rows verify plan, emitted symbolic circuit, and complete truth table for $n = 21$–30.	Bridge set is intentionally narrow because complete truth tables grow exponentially.

Table 10 makes the comparison protocol explicit as reviewer-facing questions. It is included to prevent the baseline set from being read as a single leaderboard: each layer has a usable conclusion and a stronger conclusion that the evidence does not support.

Table 10: Comparison protocol audit. Each comparison layer is tied to the reviewer question it answers, the conclusion supported by the current evidence, and the stronger claim excluded by the logical-layer experiment design.

Comparison layer	Reviewer question	Usable conclusion	Excluded conclusion
Primary same-task Boolean-oracle baselines	Is the method only compared with weak or self-written constructions?	Matched bit-flip oracle baselines support lower T-count and weighted score under the paper's logical model.	They do not prove universal CNOT, depth, ancilla, line-count, or hardware-mapped optimality.
External logic-network and LUT/XAG/AIG probes	Does the advantage survive mature external logic-synthesis toolchains?	External probes test whether the score advantage persists beyond the project's internal baselines.	They are not a full ROS SAT garbage-management flow, reversible emission, or hardware-mapped comparison.
Exact reversible-synthesis counterpoint	Is there a reversible-synthesis comparison, not only logic-network proxies?	RevKit CLI supports a genuine exact-oracle reversible-synthesis probe with lower T/score for Resource-NMCTS.	It does not imply lower auxiliary-line count, routed depth, or mapped Clifford+T cost.
Phase/R_z proxy branch	How is the RevKit phase/R_z gate-set mismatch handled?	The phase branch supports a logical R_z/phase proxy, learned affine shortlist result, and bounded sequence-smoke check.	It is not a final approximate-rotation sequence or Clifford+T decomposition.
Internal search-control ablations	Is the AI/MCTS contribution separated from the algebraic search space?	Ablations support bounded search-control, pruning, gating, and planning-time gains.	They do not show that deep reinforcement learning alone causes the main resource drop.
Scaling and correctness bridges	Does the work go beyond small truth-table functions while preserving correctness evidence?	Large rows support logical symbolic correctness and bridge-truth-table verification within the stated envelope.	They do not prove exhaustive high-dimensional truth-table optimality or benchmarking completeness.
Trade-off and non-dominance audits	Does the weighted score hide unfavorable CNOT, depth, ancilla, or runtime behavior?	The paper can claim a strong T/weighted-score point while explicitly reporting unfavorable resource dimensions.	It cannot report a single-metric score win as complete Pareto or hardware dominance.

Table 11 answers the narrower question of whether each target family is a meaningful comparison for the claim being made. It labels the comparison role explicitly: primary benchmark, external stress test, exact reversible counterpoint, phase proxy, causal control, scalability verification, or non-dominance boundary. The labels are used in the results discussion so that SSHR is not treated as the whole story, ROS and XAG/LUT/AIG flows are not treated as full hardware-mapped reproductions, and AI ablations are not treated as competing oracle synthesizers.

Table 11: Comparison target validity audit. Each target family is assigned the role it plays in the manuscript, the reason the comparison is meaningful, and the stronger conclusion that the row does not support. The corresponding CSV keeps the full method/control list for each family.

Target family	Role	Why meaningful	Invalid statement
Same-task Boolean-oracle baselines	primary benchmark	These baselines solve the same bit-flip Boolean-oracle task or the closest small-function CNOT-oriented variant under the logical resource model.	Not evidence of universal CNOT, depth, ancilla, line-count, or hardware-mapped optimality.
External logic-network probes	external stress test	These probes test whether mature Boolean-network and LUT/XAG/AIG optimization already removes the apparent advantage.	Not a full ROS SAT garbage-management optimizer, reversible-emission, routing, or hardware-mapped comparison.
Published STG optimum-library counterpoint	small-function optimum counterpoint	This target asks whether the generic logical search beats a precomputed tiny-function optimum library; the expected use is a boundary, not a headline win.	Not evidence of beating published STG T-count, T-depth, or qubit optima.
Exact reversible-synthesis probe	exact reversible counterpoint	The baseline embeds each function as the exact reversible permutation and checks a genuine reversible-synthesis toolchain.	Not evidence of lower auxiliary-line count, routed depth, or final mapped Clifford+T dominance.
Phase/R_z branch	phase proxy	This branch tests whether the search framing transfers to verified logical phase/R_z proxy objectives.	Not a final approximate-rotation synthesis or full Clifford+T sequence.
AI/search-control ablations	causal control	These controls separate representation/search-space effects from MCTS, neural ranking, guards, and budget allocation.	Not evidence that deep reinforcement learning alone causes the full resource reduction.
Scaling and correctness bridges	scalability verification	These rows test whether emitted logical oracles remain verifiable beyond the small truth-table benchmark slice.	Not exhaustive high-dimensional truth-table benchmarking or optimality evidence.
Trade-off counterpoints	non-dominance boundary	These counterpoints keep the comparison meaningful by exposing metrics where other methods remain strong.	Not a single universal leaderboard or complete Pareto dominance claim.

Table 12 gives the corresponding route map: each claim is tied to the comparator family that can answer it and to the over-claim that remains excluded, so the layered comparison set is not read as a single leaderboard.

Table 12: Comparison-route decision audit. Each row states which comparator family should be used for a reviewer-facing claim and which stronger interpretation is excluded by the current logical-layer evidence.

Route	Comparator	Use for	Do not claim
Same-task resources	ANF, ESOP, ABC, BDD, SSHR	main bit-flip resource gains	hardware or global optimality
SSHR CNOT boundary	SSHR-H or SSHR-I	CNOT counterpoint	CNOT dominance or SSHR variant
External toolchains	ROS, mockturtle, Caterpillar, CirKit	external logical stress	full ROS, routing, or hardware mapping
RevKit phase	RevKit CLI and phase R_z	reversible and phase counterpoint	line or Clifford+T dominance
Tiny optima	STG $n = 4/5$ optima	tiny optimum boundary	beating published STG optima
AI and MCTS controls	no-MCTS, random, learned controls	search attribution	deep RL only explanation
Large-scale check	$n = 20$–64 symbolic; $n = 21$–30 bridge	large-n verification	exhaustive high-n optima
Tradeoff boundary	dominance and weight profiles	score and resource tradeoff	universal Pareto dominance

Table 13 turns the role audit into a quantitative answer to what the proposed method is compared against. It combines the verified-row counts, paired score outcomes, search-control ablations, and raw four-resource dominance checks used throughout the Results section. The table is deliberately phrased as an answer scorecard rather than as a universal leaderboard: each row states the comparison target, the evidence that makes the comparison meaningful, and the claim boundary that prevents over-reading SSHR, external logic-network probes, RevKit, or learned controls.

Table 13: Comparison answer scorecard. The table gives the short answer to "compared with what?" by linking each comparison target to verified evidence, headline quantitative outcomes, and the excluded claim that keeps the logical-layer comparison meaningful.

Question	Target	Evidence	Quantitative answer	Boundary
Are gains only over weak algebraic baselines?	Direct ANF, ESOP beam/MILP, ABC/BDD exports	Internal logical baselines: 1770/1770; Exported SSHR/ABC/BDD extension: 1416/1416	Pareto vs direct ANF 172/1/4, -67.80%; vs ESOP-MILP 167/3/7, -29.84%; vs ABC-XAG 177/0/0, -65.58%	No universal CNOT, depth, ancilla, line-count, or hardware-mapped optimality claim.
Is SSHR the whole comparison target?	SSHR-H and SSHR-I CNOT	Internal logical baselines: 1770/1770; Exported SSHR/ABC/BDD extension: 1416/1416	Pareto vs SSHR-H 173/4/0, -41.06%; vs SSHR-I CNOT 173/4/0, -31.89%; SSHR-H four-resource dominance 33/4/140/0	Do not claim CNOT dominance over SSHR; SSHR remains a real CNOT counterpoint.
Does the advantage survive external logic synthesis?	ROS-style LUT/garbage-budget, mockturtle XAG, Caterpillar API, CirKit AIG/MC	ROS-style LUT proxy: 309/309 best rows; 927/927 K-sweep rows; 927/927 garbage-proxy rows; 1059/1059 garbage-budget rows; mockturtle KLUT-to-XAG probe: 241/241; Caterpillar XAG API probe: 177/177 best rows; 531/531 strategy rows; CirKit AIG/MC probe: 241/241	ROS best-K 309/0/0, -84.27%; mockturtle n≤6 166/11/0, -31.50%; Pareto vs Caterpillar API score 177/0/0, -47.94%; CNOT 4/173/0; CirKit n≤6 177/0/0, -62.34%	Not a full ROS SAT garbage-management optimizer, reversible-emission, routing, or hardware-mapped comparison.
Does it beat published tiny-function optima?	Published STG n=4/5 optimum table	Published STG optimum-library counterpoint: 270/270	Pareto vs STG T-count optimum 0/40/14, +84.26%; vs direct ANF on STG slice 50/4/0, -58.10%	No claim of beating published STG T-count, T-depth, or qubit optima on n=4/5 spectral representatives.
What does RevKit test?	Legacy RevKit exact oracle and RevKit phase/R_z branch	Legacy RevKit CLI exact oracle: 708/708; RevKit phase/R_z branch: 531/531; Learned phase pruning: 7611/7611	RevKit CLI exact oracle 173/0/4, -67.28%; Affine phase vs RevKit oracle_synth 177/0/0, -65.50%	Not lower auxiliary-line count, routed depth, or final approximate-rotation Clifford+T synthesis.
What part is actually AI/MCTS?	No-MCTS portfolio, Pareto archive, learned/random controls	Search-control audit: 10/10 pass	Resource over no-MCTS 54/0/123, -1.44%; Pareto over no-MCTS 106/0/71, -4.69%; learned vs random prior 17/8/152, -0.15%	Do not state that deep reinforcement learning alone explains the full resource reduction.
Is the evidence only small-scale?	$n = 20$–64 symbolic runs and $n = 21$–30 complete truth-table bridges	High-dimensional frontier search: 2088/2088; Complete truth-table bridges: 700/700	Stage-gated vs all-depth scale 0/4/92, +0.04% score; -25.43% time; ultra n=48,56,64 stress 480/480 plan+circuit verified; Bridge rows verify plan, emitted symbolic circuit, and complete truth table for n=21-30.	Not exhaustive high-dimensional truth-table benchmarking or global optimality proof.
Does weighted score hide bad tradeoffs?	Four-resource dominance, non-dominated pool, and resource-weight sensitivity	12-method n≤6 dominance pool; 177 matched functions; resource-weight sensitivity 13794 pair/profile rows; 13 comparisons; 6 profiles	Pareto-Resource-NMCTS is 170/177 non-dominated; RevKit CLI exact oracle four-resource dominance 3/0/170/4; CirKit AIG/MC four-resource dominance 21/0/156/0; Caterpillar CNOT-only 4/173/0 while paper score is 177/0/0	Do not turn weighted-score wins into a complete Pareto or hardware-dominance claim.

Table 14 gives the corresponding SSHR reproduction-scope audit. This table is included because SSHR is both the closest geometric small-function baseline and the easiest comparison to over-read. The audit separates source-anchored paper-reference checks, same-function SSHR-H rows, time-limited SSHR-I rows, and the exact $n \leq 4$ pilot from the stronger claim that the lightweight rebuild fully reruns every published SSHR random experiment. The results section therefore uses SSHR as a CNOT-oriented counterpoint, not as the definition of the proposed method.

Table 14: SSHR reproduction-scope audit. The table records which SSHR-facing elements are reproduced or source-anchored in the current package, and which stronger SSHR claims remain outside the logical-layer submission.

SSHR-facing element	Scope	Current evidence	Boundary
SSHR literature and method boundary	covered	The manuscript cites SSHR, states that the proposed method does not search parallelotope candidates, and uses SSHR as a CNOT-oriented small-function baseline.	The proposed method is not an SSHR variant and does not inherit SSHR's CNOT-only objective.
SSHR Table VIII candidate-space anchor	source-anchored	The development repository contains the SSHR candidate-count reference and table runner when the full source tree is present; the upload payload records the boundary without requiring that parent source tree.	This audit does not rerun every published SSHR random benchmark table inside the lightweight submission rebuild.
SSHR-H same-task baseline rows	covered	The traditional bit-flip benchmark includes 177 correct SSHR-H rows and the manuscript keeps SSHR-H visible as the best mean-CNOT baseline in the traditional table.	The SSHR-H row is not evidence of CNOT-only dominance by the proposed method.
SSHR-I CNOT/T extension rows	partial	The external n¡=6 extension includes SSHR-I CNOT-objective and T-objective rows for the same 177 functions with correctness checks and explicit timeout metadata.	The n¡=6 extension is not an exact certificate for all SSHR-I random-paper settings.
n¡=4 exact/pilot SSHR-I cross-check	covered	The n¡=4 external pilot records 72 functions and 216 SSHR-H/SSHR-I rows, and exact small-slice analyses include SSHR-I references as counterpoints.	Even this slice is a counterpoint under the paper's logical resource projection, not a proof of global reversible optimality.
SSHR tradeoff in resource-weight sensitivity	covered	The resource-weight audit explicitly keeps SSHR-H and SSHR-I CNOT-only losses visible while showing paper-score and T-score advantages.	Weighted-score wins cannot be rewritten as all-metric or CNOT-only SSHR dominance.
Comparison and claim-boundary integration	covered	Comparison answer, target-validity, threats-to-validity, and claim-scope gates all mark SSHR as a CNOT counterpoint rather than the whole comparison target.	No paper-facing text may claim full SSHR replacement, universal optimality, or hardware-mapped dominance.
Reviewer payload visibility	covered	The submission package exposes the comparison role, reviewer concern brief, and artifact guide so a reviewer can locate SSHR rows and their boundary quickly.	Support-package visibility does not add a new SSHR random-table rerun.

Table 15 makes the fairness boundary explicit. It separates comparisons that share the same bit-flip oracle target from logic-network proxies, exact reversible-oracle probes, phase/R_z proxies, and large symbolic bridge checks. The table is intentionally conservative: a comparison is used only for the conclusion supported by its task alignment, verification route, and resource abstraction.

Table 15: Baseline comparability audit. The table records why each comparison family is fair enough for the paper's bounded claim, which controls are used, and what residual abstraction mismatch remains.

Baseline family	Task alignment	Fairness control	Residual risk	Usable claim
Direct algebraic and ESOP baselines	Same bit-flip Boolean oracle target and the same logical resource model.	Matched $n \leq 6$ functions, complete truth-table checks, identical score coefficients, and paired per-function comparisons.	These baselines are representation-level constructions, not mature external compilers.	Primary evidence for lower T-count and weighted score under the paper's own oracle model.
SSHR-H and SSHR-I	Same small Boolean-function oracle setting, but with a CNOT-oriented parallelotope search objective.	Matched functions and resource extraction; SSHR-I timeout rows are separated from completed rows.	SSHR is optimized for CNOT structure and small n, so it is a counterpoint rather than the method's design parent.	Resource-NMCTS improves score and T-count while SSHR remains the strongest CNOT reference.
ABC/BDD and exported logic networks	Same exported Boolean functions, evaluated through logical AIG/XAG/ LUT/BDD resource proxies.	Rows with errors, skips, or failed truth-table checks are excluded; paired comparisons use matched names.	Network-level counts are proxies and do not include reversible garbage management or quantum routing.	Mature classical logic optimization does not by itself remove the T/score advantage.
ROS-style LUT and XAG/AIG/ API probes	Same benchmark functions pass through LUT, XAG, AIG, and Caterpillar API probes with export/readback checks where available.	The LUT proxy includes best-K, line-aware reselection, and an executable garbage-budget frontier; mockturtle, CirKit, and Caterpillar use official source/toolchain/API paths and row-level correctness checks.	These are not full ROS SAT garbage-management optimizers or hardware-mapped reversible flows.	The advantage persists beyond self-written baselines, with CirKit and Caterpillar exposing real CNOT/depth tradeoffs.
Legacy RevKit CLI exact-oracle portfolio	Each function is embedded as the exact reversible permutation $(x, y) \mapsto (x, y \oplus f(x))$.	TBS, DBS, and RMS are all run and reduced to a per-function best-score portfolio.	CNOT and depth are derived from Toffoli-control distributions; RevKit uses fewer auxiliary lines on most rows.	A genuine reversible-synthesis probe still favors Resource-NMCTS on T-count and score, but not on line count.
Phase/R_z baselines and learned shortlist	Same Boolean functions are evaluated as phase-oracle or affine-FPRM phase-search instances.	Rows are verified up to global phase; learned shortlists are checked against same-budget random controls and wide exact search.	The branch is a logical R_z/phase proxy and does not output rotation-synthesis sequences.	The search framing extends to verified phase/R_z proxies, but it is not a finished Clifford+T compiler.
High-dimensional symbolic and bridge checks	Large term-set instances and $n = 21$–30 bridge functions test the same emitted logical oracle semantics.	Symbolic ANF/circuit checks cover all reported high-dimensional rows; bridge slices add complete truth-table checks.	Complete truth-table enumeration is limited to bridge slices and generated functions.	The large-scale evidence supports semantic correctness and scaling of the logical search, not exhaustive high-dimensional optimality.

Table 16 records the strongest opposing evidence that bounds the comparison claims. This audit is included because a meaningful comparison should not hide metrics on which other methods remain strong. SSHR is therefore treated as a CNOT counterpoint, Caterpillar API as an implementation-family CNOT-pressure counterpoint, CirKit as a depth counterpoint, RevKit CLI as an auxiliary-line counterpoint, and the learned-prior rows as evidence for incremental search control rather than for a deep-RL-only explanation.

Table 16: Counterpoint and claim-boundary audit. The table reports the strongest unfavorable metric evidence that constrains the manuscript's claims, the favorable evidence that keeps each comparison meaningful, and the resulting claim boundary.

Counterpoint	Opposing evidence	Favorable evidence	Claim boundary
SSHR family CNOT optimum	vs SSHR-I CNOT: CNOT 0/168/9, +62.06%; vs SSHR-H: CNOT 43/128/6, +18.99%	score vs SSHR-I CNOT 173/4/0, -31.89%; score vs SSHR-H 173/4/0, -41.06%	Use SSHR as a small-function CNOT counterpoint; claim T-count and weighted-score advantage, not CNOT dominance.
CirKit AIG/MC depth	depth 16/156/5, +93.16%	score 177/0/0, -62.34%	Use CirKit as a depth-oriented external probe; do not claim depth dominance over logic-network synthesis.
Caterpillar API CNOT pressure	CNOT 4/173/0, +195.18%	score 177/0/0, -47.94%	Use Caterpillar as a bounded ANF-XAG implementation-family counterpoint; do not claim CNOT-only, full-ROS, or hardware-mapped dominance.
RevKit exact-oracle auxiliary lines	peak ancilla 0/169/8, +153.11%	score 173/0/4, -67.28%	Use RevKit CLI as an exact reversible-oracle probe; do not claim line-count or mapped Clifford+T dominance.
Learned prior is incremental	time 37/140/0, +18.77% versus no-prior rerun	score 29/0/148, -0.78%	Frame AI as search control, ranking, pruning, and gating; do not claim deep RL alone explains the main resource drop.
Large-n verification is bounded	complete truth-table enumeration is limited to bridge slices	symbolic scale rows 1608/1608; complete truth-table bridge rows 700/700	Claim logical correctness inside symbolic and bridge-verification envelopes, not exhaustive large-dimensional benchmarking.

The comparison set should therefore be read as layered evidence rather than as a single universal leaderboard. Direct ANF, ESOP, ABC/BDD, and related representation baselines test matched bit-flip oracle resource efficiency; SSHR tests whether the proposed search remains competitive against a CNOT-oriented geometric method on small functions; mockturtle, Caterpillar API, CirKit, ROS-style LUT, and RevKit probes test whether the advantage persists outside the project's own implementations; and internal ablations test whether the search controls contribute beyond deterministic construction. The results below use each family only for the claim it can support.

The experiments were executed as a reproducible artifact package rather than as isolated hand runs. Table 17 records the local compute envelope, manifest-level parallelism, artifact counts, and external toolchain provenance. Runtime numbers should be read within this workstation context; the logical resource counts are the portable experimental outcome.

Table 17: Compute and reproducibility audit. The table documents the workstation, parallel-run manifests, artifact coverage, and external-tool commits used by the current submission package.

Aspect	Evidence	Boundary
Host compute envelope	Apple M4 Pro; 14 physical/14 logical CPU cores; 24.0 GiB RAM; Apple M4 Pro, 20 GPU cores, Metal 4; Python 3.11.15	Workstation context for reproducibility; not a hardware-mapping result.
Learning accelerator	PyTorch 2.12.0; MPS=True; CUDA=False	GPU/MPS is available for neural training; synthesis resources remain logical-layer estimates.
Parallel-run provenance	56 manifests record worker counts; max workers=10; traditional resource 10 workers/1770 rows; RevKit CLI 8 workers/708 rows; ROS-style LUT 8 workers/927 rows; CirKit 8 workers/177 rows; mockturtle 4 workers/177 rows	Wall times are machine-dependent and are not claimed as portable speedups.
Artifact coverage	131 top-level run/train/analyze scripts; 161 raw CSVs; 250 summaries; 144 manifests; 199 paper tables; 7 figure panels (21 PDF/PNG/SVG assets)	Counts describe the current artifact package, not independent datasets.
External-tool provenance	mockturtle 25beb0e; CirKit 4531533; RevKit CLI 104eb35	Toolchain rows are logic-level probes or exact-oracle reversible probes, not full hardware flows.

Table 18 summarizes the wall-clock envelope behind the current artifact package. It is included to make feasibility and reviewer reruns easier to judge. These timings are workstation-context measurements; they are not used as portable speedup claims, and the portable claims remain the logical resources and verification status.

Table 18: Runtime-envelope audit over existing raw CSV artifacts. Rows report the checked subset, successful rows, and workstation wall-clock summaries used to bound the feasibility of the submitted experiments.

Slice	Scope	Rows ok	Runtime evidence and boundary
Traditional $n \leq 6$ Resource-NMCTS	177 matched benchmark functions	177/177	median=4.15s; p95=9.78s; max=15.44s. Small-function timing context only; headline claims are paired logical-resource improvements.
External logic-tool probes	ROS-style LUT, Caterpillar, mockturtle, CirKit, RevKit	1548/1548	median=0.08s; p95=0.51s; max=19.06s. Tool rows are bounded logic-level probes or exact-oracle probes, not hardware-mapped baselines.
High-dimensional symbolic frontier	$n = 24, 28, 32, 40$ plus $n = 48, 56, 64$	144/144	median=3.30s; p95=52.14s; max=117.72s. Symbolic emitted-circuit checks; no exhaustive truth tables outside bridge slices.
Complete truth-table bridge	$n = 21$–30 generated bridge slices	70/70	median=2.85s; p95=14.87s; max=17.02s; truth_verify median=0.19s. Complete checking is limited to generated bridge slices and is not global high-dimensional enumeration.
Learned-control runtime boundary	compressed bit-flip budgets plus promoted learned controls	708/708	median=2.12s; p95=6.41s; max=7.72s. Learned controls are quality/budget evidence with explicit overhead or savings, not a blanket speedup claim.

Table 19 records the claim-to-artifact chain for the submission package. It is not a new result table; its role is to make each headline claim auditable by pointing to the scripts, CSVs, tables, figures, and manuscript anchors that carry the evidence.

Table 19: Submission traceability audit. Each row links a paper-facing claim family to the manuscript location and artifact chain used to support it.

Claim family	Paper-facing use	Manuscript anchor	Boundary
Method formulation	Defines Resource-NMCTS as a source-anchored logical ANF/FPRM search workflow.	Method, Tables method-workflow, algorithm-contract, and search-budget-contract	Establishes executable stages, semantic/resource guarantees, and bounded search budgets, not a hardware compiler or global optimality proof.
Contribution mapping	Links each stated contribution to implementation, evidence, and claim boundary.	Introduction, Table contribution-map	Prevents the introduction from making unsupported novelty claims.
Novelty and comparison scorecard	Answers reviewer-facing questions about method identity, comparison meaning, external probes, tradeoffs, AI isolation, and scale boundary.	Introduction, Table novelty-comparison-scorecard	Provides a reviewer-facing index over existing evidence; it does not add a new metric or broaden the logical-layer scope.
Primary small-function resources	Supports T-count and weighted-score claims on matched $n \leq 6$ functions.	Results, traditional functions	Same logical cost model; not a CNOT-only or hardware-level claim.
Baseline scope and fairness	Explains why each baseline family is comparable and which claim it can support.	Experimental Design, benchmark-suite audit, baseline matrices, target-validity table, comparison route-decision table, comparison answer scorecard, SSHR reproduction-scope audit, and counterpoint audit	Treats comparisons as layered evidence with explicit roles and reports unfavorable metric counterpoints rather than a universal leaderboard.
Threats to validity	Links the main reviewer-facing validity threats to mitigation evidence and residual boundaries.	Discussion, Table threats-validity	Names limitations and mitigations; it does not remove the logical-layer, proxy-baseline, or generated-slice boundaries.
External toolchain probes	Traces ROS-style LUT, published STG, mockturtle, Caterpillar API, CirKit, and RevKit CLI comparison evidence.	Results, external probes	Logic-level probes, published optimum-library counterpoint, or exact-oracle reversible probe; not full hardware mapping.
Multi-resource tradeoff	Audits whether weighted-score wins imply raw-resource, schedule-proxy, or auxiliary-lifetime dominance.	Results, raw multi-resource dominance, score-weight robustness, resource-weight sensitivity, CNOT constraint profile, and schedule-proxy audit	Dominance and sensitivity checks use logical T-count, CNOT, depth, gates, peak ancilla, T-depth proxy, and auxiliary lifetime; no routing or native-gate scheduling is claimed.
Learned-control contribution	Separates search-control baselines and promoted learned controllers from limited diagnostics.	Results, search contribution and ablation	Supports guarded learned-control evidence, not a claim that RL alone explains all gains.
High-dimensional verification	Traces large symbolic checks through $n=64$, phase checks, and complete truth-table bridge slices.	Results, high-dimensional verification	Large rows are symbolic or bridge-slice verified, not exhaustive high-dimensional truth tables.
Runtime envelope and execution cost	Consolidates workstation timing evidence for feasibility without promoting portable runtime-speedup claims.	Experimental Design, runtime-envelope audit table	Wall-clock evidence is workstation-context feasibility evidence; portable claims remain logical resources and verification status.
Archive package integrity	Hashes stable submission payload groups and records archive boundaries.	Experimental Design, archive manifest	Excludes terminal submission audits and compiled PDF from payload hashes to avoid self-reference.
Submission support and venue gate	Traces target-venue decision support, support-packet consistency, and author-gated submission metadata.	Submission package, target venue brief and author-input packet	Supports a source-backed venue decision path; final venue, declarations, archive links, and anonymous-review status remain author input.
Reproducibility package	Documents compute envelope, worker manifests, artifact counts, raw rerun entry points, and availability section.	Experimental Design and Data and Code Availability	Repository-relative package until an archival DOI or anonymous link is supplied.

The submission archive manifest in Table 20 adds a package-level check over the stable payload groups. It hashes source files, tables, figures, raw measurements, summaries, manifests, scripts, trained models, and local tool adapters while excluding terminal submission audits and the compiled PDF from the digest set. This exclusion avoids self-referential hashes because the final PDF contains the manifest table itself; PDF existence and page count are checked separately by the readiness audit.

Table 20: Submission archive manifest. Each row reports a stable payload group, the number of files found, missing expected files, a category digest, and the scope boundary for that group.

Payload group	Files	Missing	Digest	Boundary
Manuscript source	5	0	`9db5a05ddcef`	Compiled PDF is checked by readiness audit to avoid self-referential hashes.
Paper tables	197	0	`7d657d8b1864`	Terminal submission audit tables are excluded from the stable payload digest.
Submission figures	28	0	`57c8751f0611`	Includes generated figure assets and plotted source data, not raw benchmark reruns.
Raw measurements	161	0	`fa1965cc81bb`	Raw CSVs are regenerated only by the heavier run scripts, not by the lightweight rebuild.
Derived summaries	442	0	`21102f76ce9f`	Terminal submission/package outputs are excluded so this manifest remains stable.
Run manifests	128	0	`b682a6326e96`	Submission-level terminal manifests are excluded from this digest group.
Scripts and docs	159	0	`cc8198e4bd43`	Includes reproducible entry points and top-level core implementation modules; raw sweeps still require their individual drivers.
Models	20	0	`4c5c2a1bfc19`	Includes trained local policy artifacts when present; model retraining is not part of the lightweight rebuild.
External adapters	1	0	`de5aa3b78998`	Includes local adapter source files used for external toolchain probes, not vendored tool repositories.
Submission support	17	0	`93706e6cbbe1`	Includes public package README, author-input packet, artifact guide, cover-letter, declaration, checklist, reviewer-brief, editor-screening, venue-selection, and structured metadata templates; ignored generated private submission text is excluded.

6 Results

6.1 Traditional functions

Figure 2 and Table 21 summarize the traditional $n \leq 6$ slice. Pareto-Resource-NMCTS has the lowest mean T-count and weighted score among the shown internal baselines, reducing T-count by 72.25% and score by 67.80% relative to direct ANF synthesis. SSHR-H retains the lowest mean CNOT count, so the result is not a CNOT-only dominance claim.

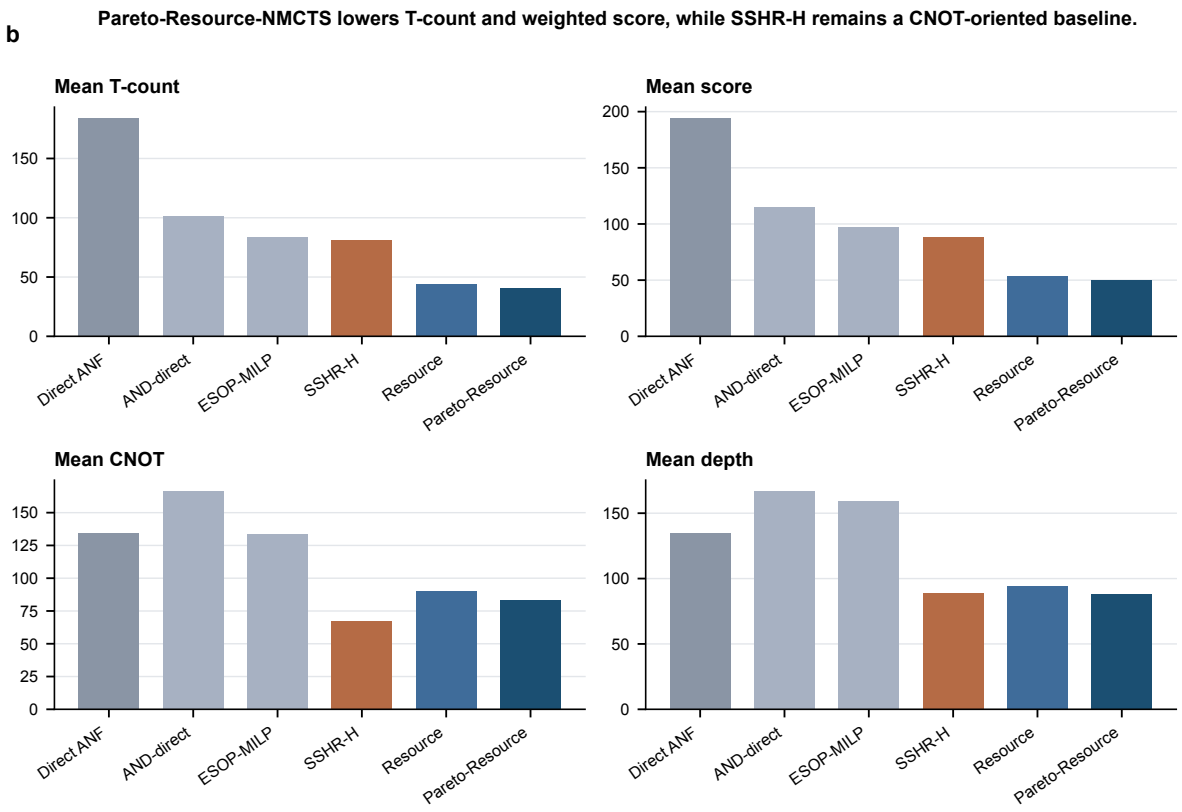

Figure 2: **Mean logical resources on the traditional slice.** Pareto-Resource-NMCTS has the lowest T-count and weighted score; SSHR-H remains a strong CNOT-oriented baseline.

Table 21: Mean logical resources on the $n \leq 6$ traditional slice.

Method	Mean T	Mean CNOT	Mean depth	Mean ancilla	Mean score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	**67.6**	88.7	1.36	88.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	**40.4**	83.0	**88.0**	2.03	**49.6**

The average improvements are not uniform across Boolean structure. Table 22 stratifies the same 177 verified functions by named family, algebraic degree, and ANF term density. The audit shows that the method should not claim an algebraic score gain on affine/parity rows, where direct ANF and ESOP already match the selected candidate. The main reductions instead concentrate on high-degree and ANF-dense rows, where factor generation, guarded candidate selection, and Pareto archiving have room to reuse nonlinear structure. Low-degree sparse rows remain a useful boundary because SSHR can still be competitive on some of them.

Table 22: Traditional-slice structure mechanism audit. Rows report paired score win/loss/tie counts and mean relative score changes for Pareto-Resource-NMCTS within structural buckets of the same verified $n \leq 6$ functions.

Structural slice	Func.	vs direct ANF	vs ESOP-MILP	vs SSHR-H	Mechanism reading
family: parity	4	0/0/4, +0.00%	0/0/4, +0.00%	4/0/0, -98.59%	trivial/affine guard: no algebraic score gain expected
family: random truth-table	160	160/0/0, -70.23%	156/2/2, -30.04%	158/2/0, -40.07%	main mechanism slice: dense high-degree ANF benefits from factor/Pareto search
family: threshold/majority	8	8/0/0, -68.47%	8/0/0, -47.17%	6/2/0, -28.50%	clear score-reduction slice
degree: degree 0-1	4	0/0/4, +0.00%	0/0/4, +0.00%	4/0/0, -98.59%	trivial/affine guard: no algebraic score gain expected
degree: degree 3	38	38/0/0, -63.35%	34/2/2, -23.46%	36/2/0, -30.60%	clear score-reduction slice
degree: degree $\geq$4	132	132/0/0, -71.72%	131/1/0, -32.29%	132/0/0, -42.69%	main mechanism slice: dense high-degree ANF benefits from factor/Pareto search
ANF terms: ANF $\leq$ n	12	7/1/4, -30.71%	7/0/5, -18.42%	10/2/0, -53.71%	low-degree boundary: SSHR can remain competitive on some rows
ANF terms: ANF ¿ 2n	124	124/0/0, -73.38%	123/1/0, -33.36%	123/1/0, -41.71%	main mechanism slice: dense high-degree ANF benefits from factor/Pareto search

6.2 External logic-network and reversible-synthesis probes

Figure 3 summarizes paired score changes against several external or traditional baselines. The ROS-style LUT proxy covers 309 functions and 927 K-sweep rows, all truth-table checked; Pareto-Resource-NMCTS wins 309/309 score comparisons against the best-K proxy. The mockturtle probe uses official headers to resynthesize KLUT networks into XAGs; Pareto-Resource-NMCTS has 166/11/0 score wins on the traditional set and 64/0/0 on the $n = 14$ set.

We further reselect the same verified LUT sweep under stricter line pressure. The minimum-ancilla selector reduces the LUT proxy's peak ancilla by 32.47% relative to the best-score proxy, but increases its own weighted score by 40.67%. Pareto-Resource-NMCTS still obtains 309/0/0 score wins against this minimum-ancilla selector and against two line-weighted selectors. This result is a robustness check for LUT-parameter choice, not a claim that the official ROS SAT garbage-management flow has been reproduced.

Table 23: Line-aware reselection of the verified ROS-style LUT proxy sweep.

Comparison	Metric	Items	W/L/T	Mean Δ
and_pareto_resource_nmcts vs external_ros_lut_min_ancilla	score	309	309/0/0	-85.83%
and_pareto_resource_nmcts vs external_ros_lut_min_ancilla	peak_ancilla	309	301/0/8	-68.21%
and_pareto_resource_nmcts vs external_ros_lut_line_w10	score	309	309/0/0	-84.45%
and_pareto_resource_nmcts vs external_ros_lut_k4_line_cap	score	309	309/0/0	-85.02%
external_ros_lut_min_ancilla vs external_ros_lut_proxy	peak_ancilla	309	197/0/112	-32.47%
external_ros_lut_min_ancilla vs external_ros_lut_proxy	score	309	0/197/112	+40.67%

We also test a reversible-garbage pressure proxy on the same best-K LUT DAGs. The proxy re-runs and re-verifies 309 selected ABC LUT networks, then compares the keep-all Bennett estimate with two recursive recomputation schedules. The zero-checkpoint schedule lowers mean peak ancilla from 6056.0 to 27.4, while raising the average logarithmic T and score costs by more than two orders of magnitude (Table 24). Thus naive recomputation exposes the line-operation trade-off that ROS-style garbage management must solve, but it does not replace the unreproduced official SAT garbage-management stage.

Table 24: ROS-style LUT garbage-management proxy over the verified best-K LUT DAGs. Logarithms are row-wise means over 309 functions; lower peak ancilla is reported together with the operation blow-up.

Policy	Rows	$\log_{10}(T+1)$	Peak ancilla	$\log_{10}(\text{score}+1)$
keep_all_bennett	309	3.29	6056.0	3.36
fanout_checkpoint	309	5.80	573.7	5.86
zero_checkpoint	309	5.93	27.4	5.99

Finally, we convert the same three executable garbage schedules into a budget-frontier view. For each verified LUT DAG and each relative line budget, we select the lowest-score feasible schedule among keep-all, fanout checkpoint, and zero checkpoint. The keep100 and minline frontiers both cover all 309 functions; minline reduces mean peak ancilla by 45.4% but shifts the mean logarithmic score by +2.61. Pareto-Resource-NMCTS still obtains 309/0/0 score wins against both keep100 and minline, and 303/0/6 peak-ancilla wins/ties against minline. This table is a ROS-facing garbage-budget stress test, not an implementation of the official ROS SAT garbage-management optimizer.

Table 25: ROS-style LUT garbage budget frontier over verified best-K LUT DAGs. The frontier selects among executable proxy schedules under relative peak-ancilla budgets; it is not the official ROS SAT garbage-management flow.

Budget	Functions	Peak cut	$\log_{10}$ score shift	Selected policies
keep100	309	+0.0%	+0.00	fanout_checkpoint:92; keep_all_bennett:217
line75	173	+71.5%	+4.38	fanout_checkpoint:173
line50	142	+79.9%	+5.23	fanout_checkpoint:138; zero_checkpoint:4
line25	126	+84.2%	+5.83	fanout_checkpoint:118; zero_checkpoint:8
minline	309	+45.4%	+2.61	fanout_checkpoint:148; keep_all_bennett:31; zero_checkpoint:130
Comparison	**Items**	**Score/T/lines**	**Mean Δ**	**Boundary**
Pareto vs keep100	309	309/0/0	-84.3%	proxy, not official ROS
Pareto vs line50	142	142/0/0	-98.7%	proxy, not official ROS
Pareto vs minline	309	309/0/0	-87.9%	proxy, not official ROS
Pareto vs minline lines	309	303/0/6	-69.4%	proxy, not official ROS

To check that the three-policy frontier is not hiding a better checkpoint choice on tractable instances, we further solve an exact checkpoint-subset subproblem on the same verified LUT DAGs whenever the number of multi-fanout checkpoint candidates is at most 12. This covers 192 DAGs, including all 177 traditional $n \leq 6$ functions. The optimizer exhaustively enumerates every checkpoint subset and then selects the lowest-score feasible schedule under the same peak-ancilla budgets. Pareto-Resource-NMCTS keeps 192/0/0 score wins against both the keep100 and minline optimized baselines, while the minline peak-ancilla comparison is 186/0/6. The remaining fanout-heavy DAGs are left outside the exact subproblem, so this is still a ROS-style reproducible subproblem rather than a full ROS SAT reproduction.

Table 26: Exact checkpoint-subset optimizer for tractable ROS-style LUT DAGs. The optimizer exhaustively enumerates checkpoint subsets for DAGs with at most 12 multi-fanout candidates. It covers all traditional $n \leq 6$ functions but does not reproduce the full ROS SAT garbage-management flow.

Budget	Funcs.	Peak	$\log_{10}$ score	Subsets	Selected policies
keep100	192	7.2	2.18	1.6	keep_all_bennett:192
line75	56	7.6	2.93	5.6	fanout_checkpoint:13; keep_all_bennett:7; zero_checkpoint:36
line50	25	8.4	2.94	12.2	checkpoint_subset:2; fanout_checkpoint:7; keep_all_bennett:7; zero_checkpoint:9
line25	9	4.2	1.76	28.9	fanout_checkpoint:1; keep_all_bennett:7; zero_checkpoint:1
minline	192	4.7	2.38	1.6	checkpoint_subset:4; fanout_checkpoint:2; keep_all_bennett:123; zero_checkpoint:63
Comparison	Items	W/L/T	Mean Δ		Boundary
Pareto vs keep100	192	192/0/0	-75.47%		exact subproblem, not full ROS
Pareto vs line50	25	25/0/0	-92.34%		exact subproblem, not full ROS
Pareto vs minline	192	192/0/0	-80.57%		exact subproblem, not full ROS
Pareto vs minline lines	192	186/0/6	-54.18%		exact subproblem, not full ROS

We also audit the local Caterpillar checkout because it is the closest open-source implementation family in the LSI ecosystem to the ROS-facing question: it exposes XAG/logic-network synthesis, single-target-gate circuit objects, and pebbling-style memory-management APIs. Table 27 records this as a source-family probe rather than a new performance baseline. The audit checks the local Git provenance, API surface, CMake build products, and a toy AIG compile/run smoke through `logic_network_synthesis`; it also verifies that no standalone Caterpillar/ROS executable is being used in the result tables. This improves the reproducibility boundary for ROS-facing comparisons while preserving the central limitation: the current submission still does not reproduce full ROS SAT garbage management.

Table 27: Caterpillar ROS-family source probe. This is a local source/API/build and toy compile smoke audit for a related open-source synthesis library; it is not a full ROS SAT garbage-management reproduction, and this source-only audit is not itself a standalone performance baseline.

Probe item	Coverage	Current evidence	Boundary
Local Caterpillar checkout	source available	source tree present; Git provenance recorded in manifest	A source checkout alone is not a reproduced ROS benchmark.
README-stated synthesis role	documented role	README tokens 4/4 for synthesis and memory-management role	This documentation role is not evidence that the paper ran full ROS.
Core API surface	API present	4 synthesis, gate, strategy, and verification header families token-checked	Header availability is not a matched benchmark result over the paper's function suite.
SAT/pebbling implementation surface	solver surface present	3 BSAT/Z3 and pebbling solver header families token-checked	These hooks are not the official ROS SAT garbage-management optimizer used as a reproduced baseline.
CMake build artifacts	partial build present	CMake caches present; 19 test objects; abcsat library present	The build artifacts do not include a standalone ROS or Caterpillar benchmark executable.
Toy AIG compile/run smoke	local compile smoke	compile return code 0; run return code 0	This is a toy API smoke test, not a published ROS reproduction or performance comparison.
Standalone baseline executable	not available	0 candidate benchmark executables detected	No standalone Caterpillar/ROS executable baseline is claimed or used in the result tables; API-adapter performance rows must remain in their separate manifest.
Manuscript and ROS-gap boundary	claim boundary explicit	paper, deliverable, and ROS-gap boundary tokens present	The manuscript must not describe the Caterpillar probe as full ROS SAT garbage management.

Having established that source-family boundary, we also run a bounded Caterpillar API performance probe on the same traditional $n \leq 6$ function names. Each truth table is converted to an ANF-XAG input, synthesized through Caterpillar's `logic_network_synthesis` API with three XAG strategies, and verified by converting the emitted circuit back to a logic network. All 531 strategy rows verify. The best Caterpillar row per function is a useful counterpoint rather than a win: it has lower CNOT/depth proxies on most rows, but much larger auxiliary-line usage, and Pareto-Resource-NMCTS obtains 177/0/0 matched score wins against it. This strengthens the external comparison while keeping the conclusion bounded to logical-layer resources and not to full ROS or hardware mapping.

Table 28: Caterpillar XAG API performance probe on the traditional $n \leq 6$ slice. The probe uses ANF-XAG inputs and Caterpillar's logic-network API; it is a verified implementation-family counterpoint, not the official ROS flow.

Caterpillar API row	Rows	Correct	Mean T	Mean score	Boundary
fast low-T	177	177	62.9153	96.5677	verified ANF-XAG input; not ROS
low-depth	177	177	62.9153	112.4941	verified ANF-XAG input; not ROS
low-T	177	177	62.9153	96.6025	verified ANF-XAG input; not ROS
best API	177	177	62.9153	96.5633	score vs Pareto 0/177/0, +102.8%; vs direct 123/54/0, -5.8%

The public STG benchmark suite is a different, deliberately stronger small-function counterpoint: it reports precomputed circuits for spectral representatives of 4- and 5-input Boolean functions, optimized separately for T-count, T-depth, and qubit count [6]. We therefore do not present it as a reproduced ROS flow or as a scalable compiler baseline. Table 29 instead records the boundary. On these 54 truth-table representatives, the published STG optima remain much

lower in T-count and line count than the current generic search, while Pareto-Resource-NMCTS still reduces the paper's own direct and AND-direct baselines on the same functions. This negative counterpoint keeps the small-function claim separate from the large-scale logical-search claim.

Table 29: Published STG benchmark counterpoint on 4- and 5-input spectral-representative truth tables. The STG rows are public precomputed optimum-library results, not a reproduced ROS SAT garbage-management flow.

Target	Baseline	T-count W/L/T	Score or line W/L/T	Boundary
Resource-NMCTS	STG T-count optimum	0/40/14, +88.58%	0/50/4, +22.17%	published optimum-library counterpoint
Pareto-Resource-NMCTS	STG T-count / qubit optima	0/40/14, +84.26%	0/50/4, +22.74%	published optimum-library counterpoint
Pareto-Resource-NMCTS	Direct ANF	50/0/4, -63.62%	50/4/0, -58.10%	same truth-table slice, same cost model
Pareto-Resource-NMCTS	AND-direct ANF	42/0/12, -36.45%	42/0/12, -33.63%	same truth-table slice, same cost model
Pareto-Resource-NMCTS	No-MCTS portfolio	12/0/42, -3.25%	19/0/35, -3.01%	same truth-table slice, same cost model

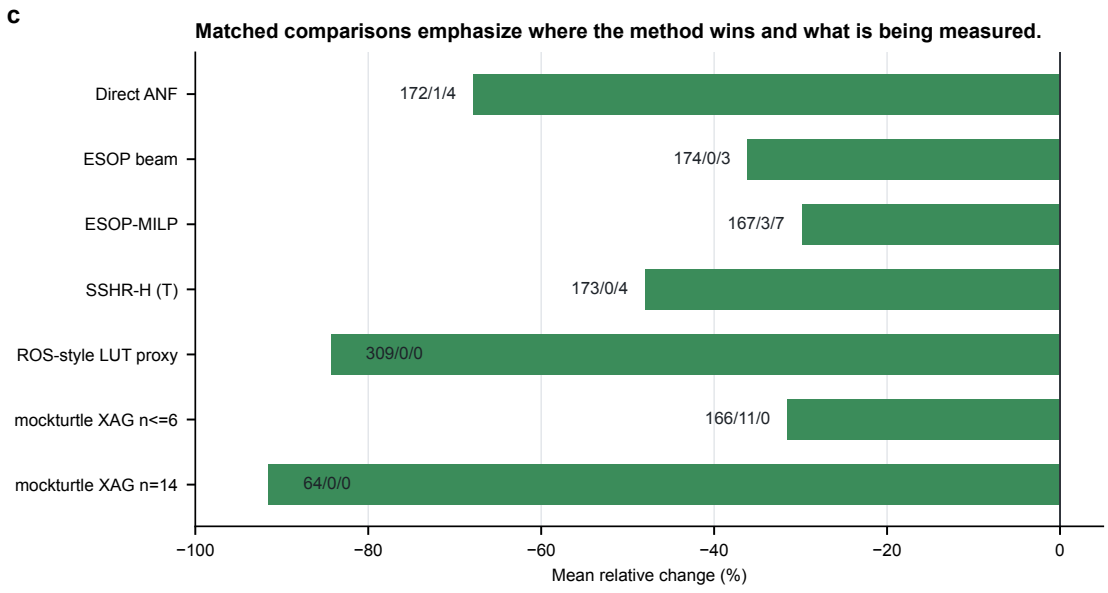

Figure 3: **Paired baseline comparisons.** Negative values favor the target method. Labels show win/loss/tie counts.

The CirKit probe converts the same BLIF benchmarks to AIGER with ABC, applies CirKit AIG rewriting and multiplicative-complexity analysis, writes Verilog, and verifies the Verilog readback through ABC. It produces 177/177 correct traditional rows and 64/64 correct $n = 14$ rows. Pareto-Resource-NMCTS wins all score comparisons but loses depth on most rows, making CirKit the strongest depth-oriented external probe in the current manuscript.

Table 30: CirKit 3 AIG/multiplicative-complexity probe on $n \leq 6$ functions.

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	177	177/0/0	-60.84%
and_pareto_resource_nmcts	177	177/0/0	-62.34%
and_fprm_polarity_archive	177	177/0/0	-58.96%
and_direct_anf	177	124/53/0	-16.74%
direct_anf	177	46/131/0	+35.23%
and_esop_milp	177	150/27/0	-39.41%
sshr_h	177	164/13/0	-32.83%

Table 31: CirKit 3 AIG/multiplicative-complexity probe on the $n = 14$ set.

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	64	64/0/0	-94.42%
and_profile_resource_nmcts	64	64/0/0	-94.42%
and_pareto_resource_nmcts	64	64/0/0	-94.46%
and_fprm_linear_pair	64	64/0/0	-94.27%
and_fprm_root_beam	64	64/0/0	-94.11%
and_direct_anf	64	64/0/0	-91.76%
direct_anf	64	64/0/0	-86.22%
and_fprm_greedy	64	64/0/0	-94.08%
and_affine_greedy	64	64/0/0	-94.08%

The RevKit CLI probe supplies a closer reversible-synthesis comparison. Each function is embedded as the exact oracle permutation $(x, y) \mapsto (x, y \oplus f(x))$, and legacy RevKit reads the SPEC permutation before running TBS, DBS, and RMS synthesis. All 531 direct flow rows return. The per-function best-score RevKit portfolio is strong on line count, but Pareto-Resource-NMCTS retains 173/0/4 score wins and a 67.28% mean score reduction.

Table 32: Legacy RevKit CLI reversible-synthesis portfolio on $n \leq 6$ functions. Each function uses the best-score result among TBS, DBS, and RMS.

Target vs RevKit CLI best-score	Items	W/L/T	Mean Δ
and_resource_nmcts	177	173/0/4	-66.32%
and_pareto_resource_nmcts	177	173/0/4	-67.28%
and_fprm_polarity_archive	177	173/0/4	-64.24%
and_direct_anf	177	167/6/4	-32.79%
direct_anf	177	87/86/4	+5.78%
and_esop_milp	177	172/1/4	-51.50%
sshr_h	177	170/7/0	+83.84%

6.3 Paired statistical evidence

The headline reductions above are paired by function or item, but mean relative improvements alone can hide skewed instances. We therefore recompute the central score comparisons directly from usable raw rows and apply an exact two-sided sign test that ignores ties. Table 33 shows that

the main traditional, external-toolchain, and high-dimensional score claims have negative median relative changes as well as strongly one-sided win counts. The high-dimensional root-beam and fast-pair margins are smaller in magnitude than the external-probe margins, but they remain consistent across paired rows.

Table 33: Paired statistical evidence for score comparisons. Rows are recomputed from correct, non-skipped raw CSV entries matched by item name. Negative mean and median changes favor the target method; $\log_{10} p$ is from a two-sided exact sign test over wins and losses, ignoring ties.

Comparison	Pairs	W/L/T	Mean Δ	Median Δ	$\log_{10} p$
$n \leq 6$ Pareto vs direct ANF	177	172/1/4	−67.80%	−71.37%	-49.54
$n \leq 6$ Pareto vs ESOP beam	177	174/0/3	−36.09%	−32.56%	-52.08
$n \leq 6$ Pareto vs ESOP-MILP	177	167/3/7	−29.84%	−23.69%	-44.96
$n \leq 6$ Pareto vs SSHR-H	177	173/4/0	−41.06%	−43.05%	-45.37
$n \leq 6$ Pareto vs SSHR-I CNOT	177	173/4/0	−31.89%	−33.91%	-45.37
$n \leq 6$ Pareto vs ABC-XAG	177	177/0/0	−65.58%	−65.25%	-52.98
ROS-style LUT best-K	309	309/0/0	−84.27%	−81.38%	-92.72
mockturtle XAG $n \leq 6$	177	166/11/0	−31.50%	−32.43%	-35.96
mockturtle XAG $n = 14$	64	64/0/0	−91.49%	−95.50%	-18.96
CirKit AIG/MC $n \leq 6$	177	177/0/0	−62.34%	−62.35%	-52.98
CirKit AIG/MC $n = 14$	64	64/0/0	−94.46%	−96.10%	-18.96
RevKit CLI exact oracle	177	173/0/4	−67.28%	−70.18%	-51.78
$n = 14$ Pareto vs root beam	64	60/0/4	−6.31%	−4.68%	-17.76
$n = 16$ Resource vs root beam	24	23/0/1	−4.36%	−3.36%	-6.62
$n = 18$ Resource vs fast pair	12	12/0/0	−3.55%	−1.79%	-3.31

To avoid treating each mean effect as a single point estimate, we also compute nonparametric bootstrap intervals on the same matched rows. This uncertainty check does not introduce new measurements; it asks whether the observed effect sizes remain separated from zero when functions or items are resampled with replacement. Table 34 shows that the main small-function and external-toolchain comparisons remain negative with narrow intervals, while the high-dimensional root-beam and fast-pair comparisons keep smaller but still negative intervals. The table therefore strengthens the paired evidence without converting a weighted-score result into a universal resource-dominance claim.

Table 34: Bootstrap uncertainty intervals for paired score-effect estimates. Rows reuse the matched pairs from Table 33. Intervals are percentile bootstrap intervals over relative score changes; negative values favor the target method.

Comparison	Pairs	Mean Δ [95% CI]	Median Δ [95% CI]	Item p10/p90
$n \leq 6$ Pareto vs direct ANF	177	−67.80% [−69.91, −65.42]	−71.37% [−73.09, −69.62]	−78.88/−54.46
$n \leq 6$ Pareto vs ESOP beam	177	−36.09% [−38.67, −33.69]	−32.56% [−35.11, −29.81]	−55.01/−19.65
$n \leq 6$ Pareto vs ESOP-MILP	177	−29.84% [−32.70, −27.04]	−23.69% [−26.68, −21.47]	−62.94/−10.26
$n \leq 6$ Pareto vs SSHR-H	177	−41.06% [−43.38, −38.77]	−43.05% [−44.08, −39.82]	−55.50/−22.11
$n \leq 6$ Pareto vs SSHR-I CNOT	177	−31.89% [−33.80, −29.90]	−33.91% [−36.68, −33.23]	−46.91/−14.30
$n \leq 6$ Pareto vs ABC-XAG	177	−65.58% [−67.10, −64.11]	−65.25% [−66.60, −63.34]	−76.73/−54.77
ROS-style LUT best-K	309	−84.27% [−85.73, −82.79]	−81.38% [−83.16, −79.32]	−99.62/−68.49
mockturtle XAG $n \leq 6$	177	−31.50% [−34.31, −28.62]	−32.43% [−33.58, −29.70]	−49.55/−9.96
mockturtle XAG $n = 14$	64	−91.49% [−94.09, −88.33]	−95.50% [−97.12, −93.60]	−98.93/−83.64
CirKit AIG/MC $n \leq 6$	177	−62.34% [−64.10, −60.59]	−62.35% [−64.66, −61.06]	−76.91/−47.69
CirKit AIG/MC $n = 14$	64	−94.46% [−95.72, −93.16]	−96.10% [−97.33, −94.71]	−98.96/−87.43
RevKit CLI exact oracle	177	−67.28% [−69.11, −65.23]	−70.18% [−71.53, −68.21]	−78.57/−57.54
$n = 14$ Pareto vs root beam	64	−6.31% [−7.89, −4.95]	−4.68% [−5.21, −3.76]	−13.44/−2.05
$n = 16$ Resource vs root beam	24	−4.36% [−6.61, −2.89]	−3.36% [−4.22, −2.46]	−6.53/−1.55
$n = 18$ Resource vs fast pair	12	−3.55% [−5.66, −1.81]	−1.79% [−4.50, −1.29]	−8.59/−1.24

6.4 Raw multi-resource dominance

Because Eq. (3) is only a ranking objective, we also test dominance without using the weighted score. A method is said to dominate another row only if it is no worse in T-count, CNOT count, logical depth, and peak ancilla, and is strictly better in at least one of them. Table 35 shows the resulting four-resource dominance counts for Pareto-Resource-NMCTS on the traditional slice. The method strongly dominates ESOP beam and many ESOP-MILP rows, but most comparisons against SSHR, CirKit, mockturtle, and RevKit are incomparable. This confirms that those baselines occupy real CNOT, depth, or line-count tradeoff points even when their weighted scores are worse.

Table 35: Raw four-resource dominance of Pareto-Resource-NMCTS on $n \leq 6$ functions. Dominance uses only T-count, CNOT count, logical depth, and peak ancilla; weighted score is not part of the predicate.

Baseline	Pairs	Pareto dom.	Base dom.	Incomp.	Equal
Direct ANF	177	69	1	103	4
ESOP beam	177	165	0	9	3
ESOP-MILP	177	123	2	45	7
SSHR-H	177	33	4	140	0
SSHR-I CNOT	177	9	3	165	0
ABC-XAG	177	33	0	144	0
mockturtle XAG	177	11	0	166	0
CirKit AIG/MC	177	21	0	156	0
RevKit CLI exact oracle	177	3	0	170	4

The same predicate can be applied to the whole method pool. Table 36 counts how often each method remains non-dominated by any other listed method on the same function. Pareto-Resource-NMCTS is non-dominated on 170/177 functions, while SSHR-I, mockturtle, and RevKit remain frequently non-dominated because they keep distinct resource tradeoffs.

Table 36: Non-dominated counts in the 12-method $n \leq 6$ pool.

Method	Rows	Non-dominated	Dominated
Pareto-Resource-NMCTS	177	170	7
SSHR-I CNOT	177	162	15
mockturtle XAG	177	159	18
RevKit CLI exact oracle	177	141	36
Resource-NMCTS	177	124	53
CirKit AIG/MC	177	56	121
SSHR-H	177	46	131
ESOP-MILP	177	44	133
ABC-XAG	177	15	162
Direct ANF	177	7	170
ESOP beam	177	6	171
AND-direct ANF	177	4	173

6.5 Search contribution and ablation

Table 37 consolidates the contribution analysis. The largest pre-portfolio gain comes from affine and Boolean-ring structural search. The learned prior gives smaller but non-degrading quality gains on the traditional slice, while the frontier policies mainly control the quality/time frontier in high-dimensional term-set experiments. Table 43 isolates the high-dimensional Boolean-ring and Boolean-screen branch. This table is deliberately mixed: it shows quality-seeking structural guards, a screen gate that preserves the selected resource vector while saving time, and a negative control where a fast screen is not resource-competitive. This prevents the structural-search claim from being reduced to a neural-prior claim or overstated as a universal screen improvement.

Table 44 then compresses the expensive depth-frontier itself. On the listed scale and truth-bridge slices, evaluating only depth 2 and depth 4 exactly matches the full depth-2/3/4 frontier while removing 24.94–30.28% of frontier evaluation time. Table 45 adds a learned gate on top of this sparse frontier. Trained on generated $n = 16, 20, 24$ term sets and applied unchanged to held-out $n = 28, 40$ term sets, it runs depth 4 on 32/48 test pairs, records zero false skips, exactly matches the sparse-frontier score, and reduces sparse-frontier evaluation time by 17.39%. The deterministic sparse frontier remains the quality reference; the learned component controls search budget after the depth-2 screen. Table 46 then audits the same trained gate without retraining on three independent seeds over $n = 24, 28, 32, 40$. Across 144 additional pairs it records zero false skips, matches the deterministic sparse frontier on every row, and saves 13.43% of sparse-frontier evaluation time. Figure 4 then sweeps the deployment threshold on the same audit. The selected threshold lies inside a zero-false-skip plateau; the best zero-false-skip point in the sweep saves 14.92%, while allowing one false skip saves 15.49% with a 0.01% mean score gap. Table 47 adds the earlier staged teacher/guard. Its depth-4 trigger is selected only on the large-policy validation split; on the independent $n = 24, 28, 32, 40$ scale rows it remains within 0.04% of all-depth score while reducing staged planning time by 25.43%.

Table 37: Matched contribution decomposition. Negative mean score changes favor the first method in each comparison.

Component	Dataset	Pairs	Score W/L/T	Mean Δ score
Affine greedy vs fixed-coordinate MCTS	ablation_affine	321	165/88/68	−12.12%
Neural refine over affine greedy	ablation_affine	322	65/0/257	−1.08%
Guarded Affine-NMCTS over no-guard	ablation_affine	322	88/0/234	−1.74%
Learned prior for Resource-NMCTS	traditional_resource	177	39/0/138	−1.10%
Pareto archive over Resource-NMCTS	traditional_resource	177	68/0/109	−3.26%
No-MCTS portfolio over heuristic-only	trad. ablation	177	54/0/123	−2.51%
Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	54/0/123	−1.44%
Pareto Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	106/0/71	−4.69%
Highdim no-MCTS portfolio over root beam	n14 ablation	16	14/0/2	−6.25%
Highdim no-MCTS portfolio over linear-pair	n14 ablation	16	14/0/2	−3.08%
Linear-pair guard vs root beam at n=14	highdim_resource	64	60/0/4	−3.00%
Recursive linear-pair guard vs root beam at n=15	n15 scale	32	30/0/2	−5.28%
Recursive linear-pair guard vs shallow linear-pair at n=16	n16 scale	24	23/0/1	−2.54%
Recursive linear-pair guard vs root beam at n=16	n16 scale	24	23/0/1	−4.31%
Baseline-preserving AI guard vs deterministic recursive guard at n=16	n16 scale	24	6/0/18	−0.05%
Fast linear-pair guard vs root beam at n=18	n18 stress	12	6/0/6	−1.91%
Resource-NMCTS vs fast linear-pair at n=18	n18 stress	12	12/0/0	−3.55%

Table 38 restates the search-control comparisons in the form used for claim interpretation. The bit-flip branch is compared against heuristic-only, beam-only, strengthened no-MCTS portfolio, MCTS, Pareto archive, and learned-prior/no-prior controls. Table 39 adds a same-budget random-prior control for the bit-flip neural scorer. The learned prior changes only action ranking, keeps the same candidate legality and semantic checks, and is compared with eight deterministic random-prior repeats per function. The effect is intentionally reported as small: learned scores are non-degrading and beat the random-prior mean, but the largest resource gains still depend on the algebraic action space and baseline-preserving guards. Table 40 then compresses the bit-flip search budget itself. With top-8 or top-12 candidate expansion and 8/12 or 12/16 MCTS simulations, the learned prior has 6/6 positive score rows across Affine-Resource-NMCTS, Resource-NMCTS, and Pareto-Resource-NMCTS against matched no-prior searches. This supports a bounded quality claim for neural budget allocation under tight search budgets, while the reported runtime overhead prevents interpreting the scorer as a speedup.

Table 38: Search-control baseline audit. Rows state which search/control baseline is being tested and which stronger claim remains outside the evidence for that comparison.

Layer	Comparison	Pairs	Score W/L/T	Mean Δ score	Claim boundary
search-space baseline	Affine greedy vs fixed-coordinate MCTS	321	165/88/68	-12.12%	This is not a neural-only effect.
deterministic search controls	No-MCTS portfolio over heuristic-only	177	54/0/123	-2.51%	This row does not isolate MCTS.
deterministic search controls	No-MCTS portfolio over beam-only	177	106/1/70	-8.36%	Beam is a child/search baseline, not the full method.
MCTS over deterministic portfolio	Resource-NMCTS over no-MCTS portfolio	177	54/0/123	-1.44%	The magnitude is incremental, not the whole resource drop.
Pareto search control	Pareto Resource-NMCTS over no-MCTS portfolio	177	106/0/71	-4.69%	Ancilla tradeoffs still need separate reporting.
learned prior	Learned prior for Resource-NMCTS	177	39/0/138	-1.10%	It is not a runtime claim and should not be promoted as the main source of improvement.
bit-flip random control	Learned bit-flip prior vs same-budget random-prior mean	177	17/8/152	-0.15%	The margin is small and runtime-negative; this is not a claim that neural ranking explains the full gain.
bit-flip low-budget control	Learned prior vs no-prior under compressed candidate/MCTS budgets	1062	218/0/844	-1.04%	The same rows still show runtime overhead, so this is budget-allocation quality evidence rather than a speedup claim.
frontier random-depth control	Large frontier policy vs same-candidate random depth	102	55/3/38; 5/0/1	-1.12%	The control is quality-positive but runtime-negative against random depth; it is not a speed, hardware-scheduling, or global-optimality claim.
high-dimensional deterministic control	Highdim no-MCTS portfolio over root beam	16	14/0/2	-6.25%	This is symbolic/high-dimensional evidence, not exhaustive truth-table optimality.
phase budget frontier	Diverse phase shortlist top-512 vs budget-32 and wide-128	38	17/0/21; 0/7/31	-2.477% vs budget-32; +0.003% vs wide-128	This is a logical phase/Rz budget-efficiency result, not a final rotation-synthesis or hardware-mapped claim.
phase random control	Diverse phase shortlist top-512 vs same-budget random mean	38	17/0/21	-0.012%	This is a phase/Rz proxy control, not a bit-flip random baseline or rotation synthesis.

Table 39: Same-budget bit-flip random-prior control. W/L/T compares the learned prior with the per-function mean over eight deterministic random-prior repeats; negative score changes favor the learned prior.

Method	Pairs	Repeats	W/L/T	Best-random wins	Mean score change	Boundary
Affine-Resource-NMCTS	177	8	20/12/145	165/177	−0.12%	same-budget random-prior scorer; lower score favors learned prior
Resource-NMCTS	177	8	17/8/152	169/177	−0.15%	same-budget random-prior scorer; lower score favors learned prior
Pareto-Resource-NMCTS	177	8	5/5/167	172/177	−0.03%	same-budget random-prior scorer; lower score favors learned prior

Table 40: Low-budget learned-prior sweep for the bit-flip branch. Learned and no-prior rows use the same functions, methods, candidate legality checks, and verification route; only the action-prior scorer and search budget differ.

Budget	Method	Pairs	Score W/L/T	Δ score	Δ time
top-8, 8/12 sim.	Affine-Resource-NMCTS	177	42/0/135	-1.28%	+36.60%
top-8, 8/12 sim.	Resource-NMCTS	177	38/0/139	-1.06%	+20.75%
top-8, 8/12 sim.	Pareto-Resource-NMCTS	177	29/0/148	-0.78%	+9.61%
top-12, 12/16 sim.	Affine-Resource-NMCTS	177	42/0/135	-1.28%	+34.21%
top-12, 12/16 sim.	Resource-NMCTS	177	38/0/139	-1.06%	+28.62%
top-12, 12/16 sim.	Pareto-Resource-NMCTS	177	29/0/148	-0.78%	+15.51%
top-24, 24/32 sim.	Affine-Resource-NMCTS	177	42/0/135	-1.47%	+91.22%
top-24, 24/32 sim.	Resource-NMCTS	177	39/0/138	-1.10%	+55.11%
top-24, 24/32 sim.	Pareto-Resource-NMCTS	177	29/0/148	-0.78%	+18.77%

Table 41 applies the same random-control idea to the large Boolean-ring frontier policy. The candidate set is fixed to the already verified depth-2/3/4 screens, and only the depth-selection policy is replaced by eight deterministic random choices. The learned policy beats the random-depth mean on held-out, independent scale, and $n = 23$ truth-bridge rows, but it spends more planning time because it selects deeper screens more often. We therefore use this control as quality-oriented evidence for learned budget allocation, not as a runtime claim. Table 42 then consolidates the repeated random-control and independent-seed evidence across the learned/search-control components. It is intentionally conservative: bit-flip prior margins remain small and runtime-positive, while the stronger stability evidence comes from frontier budget allocation, the sparse depth-4 gate, and the phase shortlist within their logical-proxy boundaries.

Table 41: Same-candidate random-depth control for the Boolean-ring frontier policy. W/L/T compares the learned large frontier policy with the per-function mean over eight deterministic random depth choices among the already verified depth-2/3/4 candidates.

Source	Scope	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Boundary
held-out frontier policy	test n=28,40	48	24/1/23	-0.78%	+59.14%	beats 8/8 random seed means; quality-oriented, not a runtime claim
scale generalization	n=24,28,32,40	96	55/3/38	-1.12%	+58.78%	beats 8/8 random seed means; quality-oriented, not a runtime claim
truth-table bridge	n=23	6	5/0/1	-0.89%	+71.00%	beats 8/8 random seed means; quality-oriented, not a runtime claim

Table 42: Stochastic learned-control stability summary. Rows consolidate same-budget random-prior, same-candidate random-depth, random shortlist, and independent-seed sparse-gate checks. Negative score changes favor the learned or guarded controller.

Component	Repeat unit	Pairs	Score W/L/T	Δ score	Robustness
Bit-flip prior	8 random-prior repeats	177	17/8/152	-0.15%	seed means beaten 8/8; ≤ best random 169/177
Pareto prior	8 random-prior repeats	177	5/5/167	-0.03%	seed means beaten 6/8; ≤ best random 172/177
Frontier policy	8 random-depth repeats	96	55/3/38	-1.12%	seed means beaten 8/8; ≤ best random 90/96
Truth bridge	8 random-depth repeats	6	5/0/1	-0.89%	seed means beaten 8/8; ≤ best random 5/6
Phase shortlist	8 random shortlists	38	17/0/21	-0.012%	seed means beaten 8/8; ≤ best random 32/38
Sparse gate	3 independent seeds	144	0/0/144	+0.00%	false skips 0; true runs 94/144

Table 43: Boolean-ring structural evidence in high-dimensional slices. Negative mean changes favor the first method in each comparison. The $n = 18$ row is kept as a speed-only negative control rather than promoted as a quality result.

Slice	Comparison	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Interpretation
$n = 14$	Boolean guard vs pairwise-wide	12	9/0/3	-0.82%	+32.80%	quality gain; slower exhaustive guard
$n = 16$	Boolean guard vs pairwise-wide	24	14/0/10	-0.34%	+22.80%	quality gain; slower guard
$n = 16$	Boolean guard vs deterministic deep	24	18/0/6	-0.52%	+356.64%	stronger quality branch; expensive
$n = 16$	Boolean-linear deep vs pairwise-wide	24	14/6/4	-0.22%	-72.31%	fast structural variant with small quality gain
$n = 20$	Resource update vs Boolean screen	6	5/0/1	-4.52%	+453.05%	quality frontier; much slower
$n = 20$	Screen gate vs full Resource	6	0/0/6	+0.00%	-75.58%	safe skip: same resources, less planning time
$n = 20$	Recursive Boolean screen vs old Resource	6	5/0/1	-7.47%	-7.32%	large-n positive screen; ancilla tradeoff
$n = 18$	Recursive Boolean screen vs old Resource	12	0/11/1	+36.94%	-97.90%	speed-only negative control; not promoted

Table 44: Sparse depth-frontier audit. The sparse controller evaluates depth-2 and depth-4 Boolean-ring screens, skips depth 3, and is compared with the full measured depth-2/3/4 frontier in the same raw files.

Source	n scope	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Depth-4 vs depth-3
scale-frontier	20,28,40	72	0/0/72	+0.00%	-27.57%	44/0/28
scale-generalization	24,28,32,40	96	0/0/96	+0.00%	-28.17%	52/0/44
truth-bridge-n23	23	6	0/0/6	+0.00%	-24.94%	5/0/1
truth-bridge-n24	24	6	0/0/6	+0.00%	-28.88%	3/0/3
truth-bridge-n25	25	6	0/0/6	+0.00%	-27.06%	4/0/2
truth-bridge-n26	26	6	0/0/6	+0.00%	-27.21%	3/0/3
truth-bridge-n27	27	6	0/0/6	+0.00%	-28.19%	3/0/3
truth-bridge-n28	28	6	0/0/6	+0.00%	-29.23%	2/0/4
truth-bridge-n29	29	6	0/0/6	+0.00%	-30.28%	3/0/3
truth-bridge-n30	30	6	0/0/6	+0.00%	-26.41%	4/0/2

Table 45: Learned sparse depth-4 gate. The gate predicts whether the depth-4 Boolean-ring screen should run after the depth-2 sparse-frontier state has been evaluated. The threshold is selected on validation data with a conservative false-skip guard and is applied unchanged to held-out $n = 28, 40$ test rows.

Split	Pairs	Run d4	False skips	Score W/L/T vs sparse	Mean Δ score	Mean Δ time	Score W/L/T vs depth-2
train	96	78	0	0/0/96	+0.00%	-10.17%	75/0/21
valid	48	38	0	0/0/48	+0.00%	-12.01%	34/0/14
test	48	32	0	0/0/48	+0.00%	-17.39%	23/0/25

Table 46: Independent-seed sparse depth-4 gate audit. The trained gate and its validation-selected threshold are reused without retraining on three new seeds. The table reports aggregate and per-n comparisons against the deterministic sparse depth-2/4 frontier.

Scope	Pairs	Run d4	False skips	Score W/L/T vs sparse	Mean Δ score	Mean Δ time	Score W/L/T vs depth-2
all	144	106	0	0/0/144	+0.00%	-13.43%	94/0/50
n=24	36	29	0	0/0/36	+0.00%	-9.88%	27/0/9
n=28	36	27	0	0/0/36	+0.00%	-12.98%	24/0/12
n=32	36	23	0	0/0/36	+0.00%	-18.19%	21/0/15
n=40	36	27	0	0/0/36	+0.00%	-12.66%	22/0/14

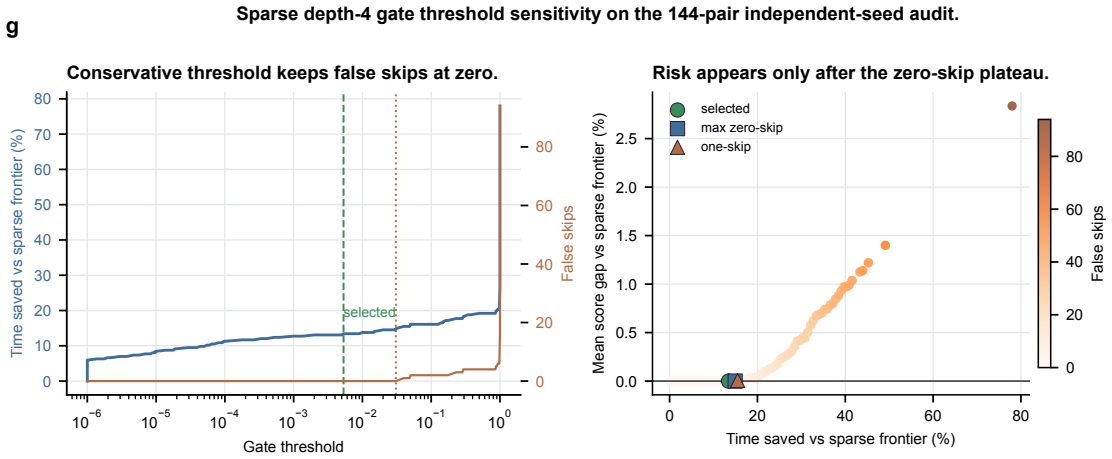

Figure 4: **Sparse depth-4 gate threshold sensitivity.** The sweep reuses the trained gate on the 144-pair independent-seed audit. The validation-selected threshold lies inside a zero-false-skip plateau, showing that the gate acts as a calibrated search-budget controller rather than as an unguarded depth-4 skip.

Table 47: Stage-gated depth-frontier controller. The depth-4 trigger is chosen on validation data and applied unchanged to scale and truth-bridge rows.

Source	Baseline	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Run depth-4
scale_generalization	adaptive_all_depth	96	0/4/92	+0.04%	-25.43%	52
scale_generalization	screen_depth2	96	58/0/38	-2.40%	+893.25%	52
scale_generalization	depth_frontier_policy	96	5/3/88	-0.06%	+101.10%	52
truth_bridge_n23	adaptive_all_depth	6	0/0/6	+0.00%	-11.51%	5
truth_bridge_n23	screen_depth2	6	5/0/1	-2.47%	+1322.09%	5
truth_bridge_n23	depth_frontier_policy	6	1/0/5	-0.12%	+94.20%	5

Table 48 reports a compact schedule-proxy view of the same high-dimensional frontier family. The metrics are computed from the emitted logical X/CNOT/MCT circuits before hardware mapping. Lower values are better. Relative to the cheap depth-2 screen, the frontier policy reduces score and T-depth proxy but increases explicit auxiliary lifetime area. Relative to the full all-depth frontier, it accepts a small score/T-depth gap while reducing auxiliary lifetime area. This is the intended resource-constrained interpretation: the learned controller moves along a quality–search-budget– auxiliary-lifetime frontier rather than claiming hardware-scheduled depth dominance.

Table 48: Logic-level schedule-proxy audit. Entries show win/loss/tie counts and mean relative change for the first method versus the second. The metrics are backend-relevant logical proxies only; they are not hardware mapping, routing, or native-gate scheduling results.

Evidence slice	Comparison	Score	T-depth proxy	Aux. lifetime area	Boundary
$n = 24, 28, 32, 40$ scale	learned frontier vs cheap depth-2	40/0/56; -1.85%	40/0/56; -1.85%	0/40/56; +20.09%	quality gain with more auxiliary lifetime
$n = 24, 28, 32, 40$ scale	learned frontier vs all-depth ceiling	0/19/77; +0.61%	0/19/77; +0.55%	19/0/77; -5.42%	near-ceiling quality with lower lifetime
$n = 21, 22$ bridge	complete-truth bridge vs cheap depth-2	8/0/4; -3.50%	8/0/4; -3.32%	0/8/4; +32.93%	truth-checked quality gain with lifetime cost
$n = 21, 22$ bridge	complete-truth bridge vs all-depth ceiling	0/2/10; +0.32%	0/2/10; +0.32%	2/0/10; -4.01%	small score gap with lower lifetime
$n = 23$ bridge	larger truth bridge vs cheap depth-2	4/0/2; -1.88%	4/0/2; -1.69%	0/4/2; +29.49%	quality gain with lifetime cost
$n = 23$ bridge	larger truth bridge vs all-depth ceiling	0/1/5; +0.61%	0/1/5; +0.52%	1/0/5; -5.09%	small score gap with lower lifetime
$n = 24, 28, 32, 40$ scale	all-depth ceiling vs cheap depth-2	58/0/38; -2.44%	58/0/38; -2.37%	0/58/38; +27.81%	quality ceiling increases lifetime
$n = 23$ bridge	bridge all-depth ceiling vs cheap depth-2	5/0/1; -2.47%	5/0/1; -2.20%	0/5/1; +36.83%	truth-checked ceiling increases lifetime

Table 49 separates promoted learned-control components from bounded controls and limited diagnostics. The frontier policies, sparse depth-4 gate, phase shortlist, and low-budget bit-flip prior provide the strongest paper-facing learned-control evidence. The root-action neural component is reported only as a bounded candidate-extension signal: it preserves the deterministic heuristic top-4 root window and adds learned top-12 actions, giving a combined 8/0/25 score W/L/T diagnostic over the heuristic root window on the audited $n = 14$ and $n = 16$ slices. The Boolean-neural guard and the generic bit-flip prior remain limited diagnostics because their quality gains are small and their runtime overhead is visible.

Table 49: Learned-control evidence audit. Rows marked as limited are reported to bound the role of AI rather than promoted as headline improvements.

Component	Class	Scope	Quality evidence	Cost evidence	Paper role
Depth-frontier policy	promoted	held-out $n = 28, 40$; 48 rows	vs oracle frontier 0/3/45, +0.04%	-51.30% time vs all-depth frontier	promoted quality/time selector
Frontier random-depth control	bounded	held-out/scale/$n = 23$; 8 random-depth repeats	scale 55/3/38, -1.12%; $n = 23$ 5/0/1, -0.89%; seed means beaten 8/8	+58.78% scale planning time vs random-depth mean	quality-oriented budget allocation; not runtime claim
Stage-gated frontier	promoted	independent $n = 24, 28, 32, 40$; 96 rows	vs all-depth 0/4/92, +0.04%	-25.43% staged planning time	promoted validation-calibrated guard
Sparse depth-4 gate	promoted	multi-seed $n = 24, 28, 32, 40$; 144 pairs	vs sparse frontier 0/0/144, +0.00%; false skips 0	-13.43% time vs sparse frontier	promoted budget gate after depth-2
Rank-diverse phase shortlist	promoted	held-out $n = 6$ phase search; 38 rows	vs budget-32 17/0/21, -2.48%; vs wide-128 0/7/31, +0.00%	512/8192 exact forms; 93.75% fewer vs wide-128	promoted phase-search pruning
Bit-flip learned prior	limited	177 n¡=6 functions; 8 random-prior repeats	vs random mean 17/8/152, -0.15%; seed means beaten 8/8	+48.05% runtime vs random-prior mean	limited quality signal, not runtime claim
Bit-flip low-budget prior	bounded	top-8/top-12 budgets; 6 score rows; 1062 pairs	learned vs no-prior 218/0/844, -1.04%	+24.22% runtime vs no-prior	bounded low-budget quality evidence, not speed claim
Boolean neural guard	limited	$n = 16$ high-dimensional guard; 24 rows	vs deterministic 4/0/20, -0.12%	+94.49% runtime	limited quality guard, not runtime claim
Root-action neural candidate extension	bounded	$n = 14$ and $n = 16$ root-action slices; 33 pairs	union top-4+neural12 vs beam4 8/0/25, -0.08%; oracle24 headroom -0.10%	root-only one-step audit; runtime not claimed	bounded candidate-extension evidence

Table 50 then calibrates the title-level *neural* MCTS framing against the evidence above. Its role is not to add another performance number, but to state which interpretation is licensed by the ablations and which stronger reading is excluded. In particular, the method can be described as a neural MCTS framework because learned policies rank, gate, or allocate bounded search budgets inside a verified algebraic synthesis workflow; it should not be described as a deep-learning-only, hardware-mapped, or globally optimal compiler.

Table 50: Neural/MCTS claim-calibration audit. Each row connects a title-level or abstract-level phrase to the generated evidence gate that supports it and to the stronger claim that remains outside the paper.

Claim anchor	Evidence gate	Allowed interpretation	Excluded interpretation
Neural component in the title	learned-control rows=9, promoted=4, bounded=3, limited=2, not_promoted=0	Neural policies and learned gates are bounded ranking, pruning, or budget-allocation controls with promoted and limited evidence classes.	Do not claim that deep learning alone explains the resource reduction.
MCTS component in the title	search-control rows=12, needs_revision=0	MCTS and Pareto search add measured, non-degrading increments over strengthened deterministic portfolios.	Do not attribute the full improvement to MCTS alone.
Causal search-control isolation	bit-flip random rows=18, budget-sweep raw rows=2124, frontier random rows=3	Same-budget, low-budget, and same-candidate controls support ranking and budget-allocation claims.	Do not use random-control rows as speed, hardware-scheduling, or deep-RL-only evidence.
Resource-constrained objective	schedule-proxy rows=8, needs_revision=0	The score and schedule-proxy evidence support resource-constrained logical-layer optimization with visible tradeoffs.	Do not turn weighted-score wins into all-resource or hardware-mapped dominance.
Verified workflow rather than learned correctness	algorithm rows=7, budget rows=8	Learning controls action order, pruning, or budget; correctness comes from GF(2), emitted-circuit, truth-table, and phase verifiers.	Do not imply that a neural model certifies circuit semantics.
Large-scale claim boundary	ultra stress rows=11, profile rows=20	Large instances are supported by symbolic term-set/emitted-circuit checks and smaller complete truth-table bridge slices.	Do not claim exhaustive truth-table benchmarking or optimality for all large n.
Logical-layer boundary	claim-scope unresolved=0, threat rows=7	The paper is a logical-layer synthesis study.	Do not claim routing, native-gate scheduling, noise, or magic-state-factory resources.

Figure 5 visualizes the same conservative separation. Promoted controllers either preserve the reference score within a small tolerance or improve it, while reducing the measured planning time or exact-evaluation budget. The two limited diagnostics fall outside this quality–effort pattern and are therefore kept out of the headline claim.

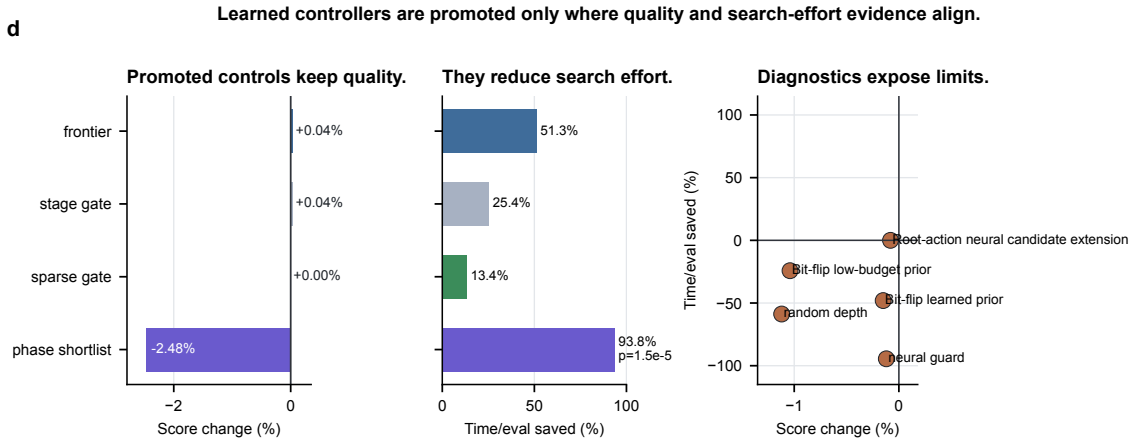

Figure 5: **Learned-control summary.** Promoted learned controllers are reported only where quality and search-effort evidence align. Positive time/evaluation saving means fewer frontier evaluations, less staged planning time, or fewer exact-scored phase candidates; negative score change favors the learned controller.

6.6 Score-weight robustness

The active score in Eq. (3) is intentionally a logical-resource ranking objective rather than a physical cost model. To check that the main comparisons are not artifacts of one coefficient choice, we recompute scores post hoc on verified rows under T-only, CNOT/depth-heavy, and ancilla-tight profiles. Table 51 shows that the small-function advantages over ESOP and SSHR-H and the high-dimensional advantages over root-beam or fast-pair guards survive these alternative profiles, while the CNOT/depth and ancilla-tight profiles expose the expected narrower margins.

Table 51: Post-hoc robustness to alternative logical-resource weights. Each cell reports score win/loss/tie counts and mean relative score change; negative values favor Resource-NMCTS or Pareto-Resource-NMCTS.

Comparison	Paper score	T-only	CNOT-depth	Ancilla-tight
$n \leq 6$ traditional: ESOP cube beam	174/0/3, -36.09%	174/0/3, -38.95%	174/0/3, -32.08%	174/0/3, -32.75%
$n \leq 6$ traditional: ESOP-MILP	167/3/7, -29.84%	166/0/11, -32.77%	165/4/8, -25.44%	167/3/7, -26.68%
$n \leq 6$ traditional: SSHR-H	173/4/0, -41.06%	173/0/4, -47.93%	168/9/0, -27.87%	172/5/0, -31.95%
$n = 14$ random ANF: FPRM root beam	60/0/4, -6.31%	60/0/4, -7.65%	60/0/4, -5.35%	56/4/4, -3.28%
$n = 16$ random ANF: FPRM root beam	23/0/1, -4.36%	23/0/1, -4.70%	23/0/1, -4.28%	23/0/1, -3.43%
$n = 18$ random ANF: FPRM root beam	12/0/0, -5.29%	12/0/0, -6.19%	12/0/0, -4.54%	11/1/0, -3.33%
$n = 18$ random ANF: fast pair guard	12/0/0, -3.55%	12/0/0, -3.75%	12/0/0, -3.23%	12/0/0, -3.33%

We also repeat this sensitivity check over the external comparison rows most likely to be questioned by reviewers. Table 52 keeps the CNOT-only profile visible: Pareto-Resource-NMCTS does not dominate SSHR-H or SSHR-I on their intended CNOT objective, Caterpillar API is a strong CNOT-only counterpoint, and mockturtle is competitive under pure T-count. The same table shows that the paper score, CNOT/depth-heavy, and ancilla-tight profiles remain favorable against the broader ABC, ROS-style LUT, RevKit, mockturtle, Caterpillar API, CirKit, and high-dimensional root-beam probes. This is therefore evidence for a bounded logical-resource search claim, not a single-metric or all-resource leaderboard.

Table 52: Resource-weight sensitivity over internal and external baselines. Each cell reports score win/loss/tie counts and mean relative score change after recomputing the matched rows under the listed logical-resource profile; negative values favor Resource-NMCTS or Pareto-Resource-NMCTS.

Baseline	Paper	T-only	CNOT-only	CNOT-depth	Ancilla-tight
Direct ANF	172/1/4; -67.8%	172/0/5; -72.3%	164/9/4; -34.2%	172/1/4; -59.5%	172/1/4; -61.0%
ESOP-MILP	167/3/7; -29.8%	166/0/11; -32.8%	123/41/13; -15.0%	165/4/8; -25.4%	167/3/7; -26.7%
SSHR-H	173/4/0; -41.1%	173/0/4; -47.9%	43/128/6; +19.0%	168/9/0; -27.9%	172/5/0; -32.0%
SSHR-I	173/4/0; -31.9%	168/1/8; -39.5%	0/168/9; +62.1%	156/21/0; -14.9%	168/9/0; -22.5%
ABC-XAG	177/0/0; -65.6%	177/0/0; -55.7%	175/2/0; -40.3%	177/0/0; -60.9%	177/0/0; -77.5%
ROS best-K	309/0/0; -84.3%	306/0/3; -84.2%	309/0/0; -82.1%	309/0/0; -83.2%	309/0/0; -83.1%
ROS min-line	309/0/0; -87.9%	306/0/3; -88.0%	309/0/0; -86.3%	309/0/0; -87.0%	309/0/0; -86.4%
RevKit	173/0/4; -67.3%	173/0/4; -72.6%	107/63/7; -9.3%	173/0/4; -56.5%	173/0/4; -58.5%
mockturtle	166/11/0; -31.5%	59/70/48; +5.7%	37/139/1; +18.4%	163/14/0; -26.3%	177/0/0; -59.8%
Caterpillar	177/0/0; -47.9%	169/0/8; -36.6%	4/173/0; +195.2%	174/3/0; -34.3%	177/0/0; -64.8%
CirKit	177/0/0; -62.3%	177/0/0; -52.9%	167/9/1; -35.0%	177/0/0; -57.0%	177/0/0; -74.7%
$n = 14$ root	60/0/4; -6.3%	60/0/4; -7.7%	60/0/4; -5.5%	60/0/4; -5.4%	56/4/4; -3.3%
$n = 16$ root	23/0/1; -4.4%	23/0/1; -4.7%	23/0/1; -4.4%	23/0/1; -4.3%	23/0/1; -3.4%

Post-hoc reweighting alone does not show whether the search policy itself responds to a

changed resource constraint. We therefore rerun the small-function resource sweep with a pure CNOT objective. The CNOT-only profile lowers the selected CNOT count for Resource-NMCTS, Profile-Resource-NMCTS, and Pareto-Resource-NMCTS relative to their balanced-profile runs, but it still does not overtake SSHR-H. Table 53 makes this distinction explicit: the proposed search has measurable CNOT sensitivity, yet SSHR-H remains the CNOT-specialized counterpoint.

Table 53: CNOT-only resource-profile rerun on the small-function sweep. The profile changes the search objective before synthesis rather than only recomputing scores afterward. Δ reports the same-method mean CNOT change relative to the balanced-profile run; negative values mean the CNOT-only rerun selected lower-CNOT circuits.

Method	Mean CNOT	Δ vs balanced	vs SSHR-H	CNOT wins
Pareto-Resource-NMCTS	71.4	-3.03%	14/33/0	14
Profile-Resource-NMCTS	76.2	-5.54%	12/35/0	11
Resource-NMCTS	77.0	-5.84%	12/35/0	11
SSHR-H	64.6	0.00%	0/0/47	33

6.7 Phase/Rz search

RevKit Python `oracle_synth` returns on all 177 traditional functions, but 171 rows contain non-Clifford R_z rotations, with 9242 such rotations in total. We therefore treat it as a phase/Rz lower-bound boundary rather than as an exact Clifford+T T-count comparison. The internal phase-parity emitter, fixed-polarity FPRM search, and affine FPRM phase search turn this boundary into a verifiable search branch.

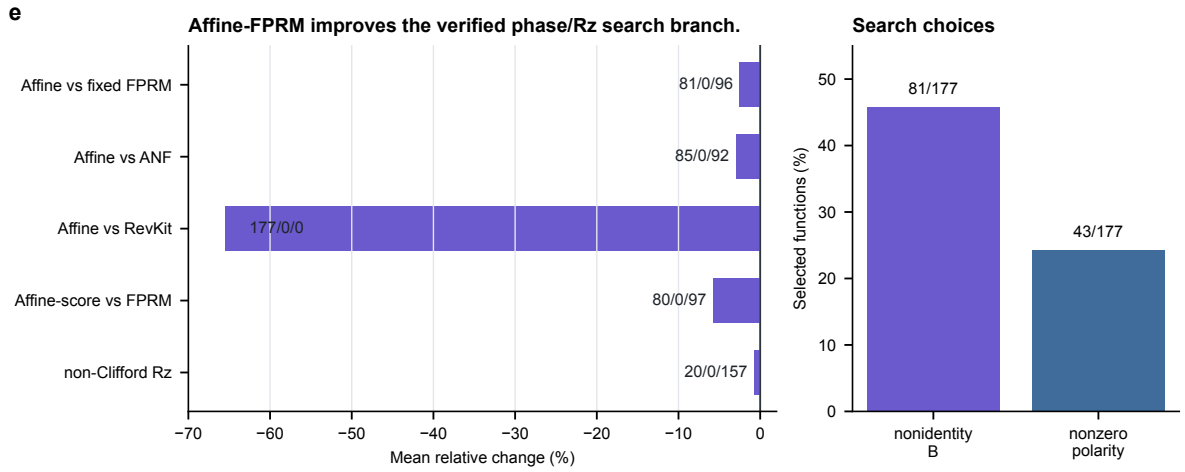

Figure 6: **Affine-FPRM phase search.** Affine preconditioning improves the phase-parity proxy over fixed-polarity FPRM and ANF baselines, while remaining a logical phase/Rz proxy rather than an approximate-rotation sequence.

Table 54: Affine-FPRM phase-parity search comparisons.

Comparison	Items	W/L/T	Mean Δ
Affine-FPRM-$T/R_z = 30$ vs FPRM	177	81/0/96	-2.51%
Affine-FPRM-$T/R_z = 30$ vs ANF	177	85/0/92	-2.98%
Affine-FPRM-$T/R_z = 30$ vs RevKit	177	177/0/0	-65.50%
Affine-FPRM non-Clifford Rz vs FPRM	177	20/0/157	-0.73%
Affine-FPRM-score vs FPRM	177	80/0/97	-5.77%

Increasing the affine transform budget from 32 to 128 yields additional but bounded gains: under the $T/R_z = 30$ proxy, the wide search gives 43/0/134 wins/losses/ties over budget 32 and reduces mean synthesis score by 0.60%. A rank-trained policy and a deterministic diversity reranker then compress this search. On the held-out $n = 6$ split, the diverse top-512 policy exact-scores only 512 of 8192 affine forms per function, keeps the budget-32 comparison at 17/0/21 with a 2.48% mean reduction, and matches the wide-128 mean within 0.01%. Table 56 makes the budget frontier explicit: diverse top-256 is already within 0.012% of wide-128 while exact-scoring only 256/8192 forms, and diverse top-512 reduces exact scoring by 93.75% relative to wide-128 while retaining a 17/0/21, 2.477% advantage over budget-32. Table 60 then uses the eight same-budget random repeats as a control. The independent unit is the held-out function: random repeats are averaged per function before the sign test. The strongest row is the diverse top-512 policy, which is 17/0/21 against the per-function random mean, has sign-test $p = 1.53 \times 10^{-5}$, and beats all eight random seed means. The absolute margins are small because the affine candidate space is dense, so this supports learned pruned phase-search feasibility rather than a claim that the current scorer solves phase/Rz synthesis globally.

To check that this conclusion is not an artifact of the single $T/R_z = 30$ constant, Table 57 freezes the same verified phase candidates and recomputes scores under a precision-sensitive logical model. Each non-Clifford R_z is charged $\lceil 3 \log_2(1/\epsilon) \rceil$ T gates, following the standard asymptotic scaling for ancilla-free Clifford+T Z-rotation approximation [13], with the same charge added to logical depth and gate count. At $\epsilon = 10^{-6}$, Affine-128 remains 177/0/0 against RevKit `oracle_synth` with a 66.12% mean score reduction; diverse top-512 remains 17/0/21 against budget-32 with a 2.38% mean score reduction and stays within 0.002% of wide-128. This is a stronger sensitivity check than a single constant proxy, but it is still a cost model rather than a high-precision rotation-synthesis result. To make this boundary executable rather than only verbal, Table 58 extracts the 20 most frequent non-Clifford/non-T-like R_z angles that occur in the verified affine phase spectra and runs a bounded internal matrix beam search over $\{H, S, S^\dagger, T, T^\dagger, X, Z\}$. The emitted strings are checked with the global-phase-invariant error $\sqrt{1 - |\mathrm{Tr}(U_{\mathrm{seq}}^\dagger U_{\mathrm{target}})|/2}$. All 20 source-derived targets pass a coarse $\epsilon_{\mathrm{smoke}} = 0.125$ sequence test, while five pass $\epsilon = 0.05$. This closes part of the Rz evidence gap, but the bounded beam is not Ross–Selinger gridsynth, does not certify optimal T count, and is not used for the headline resource claims. Table 59 makes the backend boundary machine-checkable: no command-line gridsynth-style tool or Python synthesis SDK is present in the recorded environment, so the internal beam smoke remains the only emitted-sequence backend and the 15 tight-tolerance failures are reported as a target set for future high-precision synthesis work.

Table 55: Affine-FPRM phase-search budget expansion.

Metric	Pairs	W/L/T	Mean Δ
score target	177	38/0/139	−1.59%
score+1/Rz target	177	40/0/137	−0.93%
$T/R_z = 30$ target	177	43/0/134	−0.60%
non-Clifford Rz	177	3/0/174	−0.29%
total Rz	177	35/2/140	−2.39%
CNOT	177	37/2/138	−1.74%
depth	177	41/2/134	−1.93%

Table 56: Exact-scoring budget frontier for learned phase candidate pruning. Forms/fn. reports shortlisted exact phase-score evaluations per held-out function relative to the wide-128 search; negative Δ values favor the learned shortlist.

Policy	k	Eval/fn.	Cut vs wide	vs B32	ΔB32	ΔW128
Rank	256	256/8192	96.88%	15/3/20	-2.389%	+0.141%
Rank	512	512/8192	93.75%	17/0/21	-2.397%	+0.133%
Diverse	128	128/8192	98.44%	15/3/20	-2.388%	+0.143%
Diverse	256	256/8192	96.88%	16/2/20	-2.469%	+0.012%
Diverse	512	512/8192	93.75%	17/0/21	-2.477%	+0.003%

Table 57: Precision-sensitive logical cost audit for the phase/Rz branch. Scores are recomputed after charging each non-Clifford R_z by $\lceil 3\log_2(1/\epsilon)\rceil$ T gates. Negative Δ values favor the target method. This table is a sensitivity model, not rotation-sequence synthesis.

Scope	Comparison	ϵ	T/R_z	W/L/T	Δ
trad.	Affine-128 vs RevKit oracle_synth	0.001	30	177/0/0	-65.795%
trad.	Affine-128 vs RevKit oracle_synth	1e-06	60	177/0/0	-66.118%
trad.	Affine-128 vs Affine-32	1e-06	60	43/0/134	-0.583%
policy	Diverse-512 vs Budget-32	1e-06	60	17/0/21	-2.381%
policy	Diverse-512 vs Wide-128	1e-06	60	0/7/31	+0.002%
policy	Diverse-256 vs Wide-128	1e-06	60	0/9/29	+0.007%
trad.	Affine-128 vs RevKit oracle_synth	1e-09	90	177/0/0	-66.226%
policy	Diverse-512 vs Wide-128	1e-09	90	0/7/31	+0.001%

Table 58: Source-derived Clifford+T rotation-sequence smoke audit. Support is the number of occurrences of the target angle in the verified affine phase spectra inspected by the audit. Error is measured up to global phase for the emitted one-qubit sequence. These rows demonstrate coarse sequence availability only; they are not optimal rotation synthesis.

Target	Angle/π	Support	Len.	T	Error
freq01-den8-15	15/8	1155	11	6	0.1119
freq02-den8-1	1/8	1088	14	8	0.1119
freq03-den32-57	57/32	1015	17	9	0.0347
freq04-den32-61	61/32	991	0	0	0.1040
freq05-den32-19	19/32	986	11	6	0.1040
freq06-den32-59	59/32	905	23	13	0.1040
freq07-den32-3	3/32	904	18	10	0.1040
freq08-den32-1	1/32	903	5	2	0.0347
freq09-den16-31	31/16	784	0	0	0.0694
freq10-den32-63	63/32	775	5	2	0.0347
freq11-den32-13	13/32	741	10	6	0.1040
freq12-den32-17	17/32	730	8	4	0.0347
freq13-den16-1	1/16	729	5	2	0.0694
freq14-den32-15	15/32	726	1	0	0.0347
freq15-den8-3	3/8	714	12	5	0.1119
freq16-den16-3	3/16	714	6	3	0.0694
freq17-den16-29	29/16	706	1	1	0.0694
freq18-den32-11	11/32	680	23	15	0.1040
freq19-den32-21	21/32	664	14	9	0.1040
freq20-den8-13	13/8	630	12	5	0.1119

Table 59: Rotation-synthesis backend availability and remaining sequence gap. This audit records which high-precision synthesis backends are available in the reproducibility environment and separates the internal coarse sequence smoke from missing Ross–Selinger/gridsynth-style reproduction.

Item	Availability	Evidence	Boundary
Internal source-derived sequence smoke	internal matrix beam	smoke=20/20; tight=5/20; max error=0.11185337158122222	This does not certify high-precision or optimal rotation synthesis.
External command-line high-precision backends	none available	gridsynth=missing; newsynth=missing; pgridsynth=missing	Missing command-line backends mean the current package cannot claim a reproduced Ross–Selinger/gridsynth flow.
Python synthesis SDK backends	none available	pyzx=missing; qiskit=missing; cirq=missing	The internal beam smoke remains the only sequence emitter unless one of these backends is added and audited.
Tight-tolerance diagnostic	5/20 tight pass	15 tight failures listed in CSV	The phase/Rz branch is still a logical proxy and should not be used as a final Clifford+T compiler claim.

Table 60: Same-budget random-repeat control for the learned phase shortlist. Each row compares a learned top-k shortlist with the mean of eight random top-k shortlists on each held-out $n = 6$ function before applying a paired sign test. Negative Δ mean favors the learned shortlist.

Policy	Top-k	W/L/T	Sign p	Mean target	Mean random	Δ mean
Rank	64	16/3/19	0.0044	1482.48	1482.80	-0.021%
Diverse	64	16/3/19	0.0044	1482.40	1482.80	-0.027%
Rank	128	14/5/19	0.0636	1482.29	1482.52	-0.015%
Diverse	128	14/5/19	0.0636	1482.15	1482.52	-0.025%
Rank	256	12/6/20	0.2379	1482.14	1482.32	-0.012%
Diverse	256	15/3/20	0.0075	1482.06	1482.32	-0.018%
Rank	512	14/3/21	0.0127	1482.03	1482.13	-0.007%
Diverse	512	17/0/21	1.53×10^{-5}	1481.95	1482.13	-0.012%

6.8 High-dimensional verification

Figure 7 summarizes the current verification envelope. The manuscript-level evidence includes 6120/6120 item-level symbolic runs for $n = 20$–64, 1512/1512 width-probe symbolic runs, 531/531 exact phase-search rows, 7611/7611 phase-policy pruning rows, and 700/700 complete truth table bridge method rows. The complete bridge spans $n = 21$–30 and checks the oracle relation, the expanded ANF plan, and the emitted-circuit symbolic semantics. The additional $n = 48, 56, 64$ stress slice keeps the same symbolic plan and emitted-circuit checks, but is not a complete truth-table benchmark. Table 61 makes the large-scale evidence explicit by separating generated functions or settings from method rows and by reporting representative resource means only for the paper-facing method in each slice.

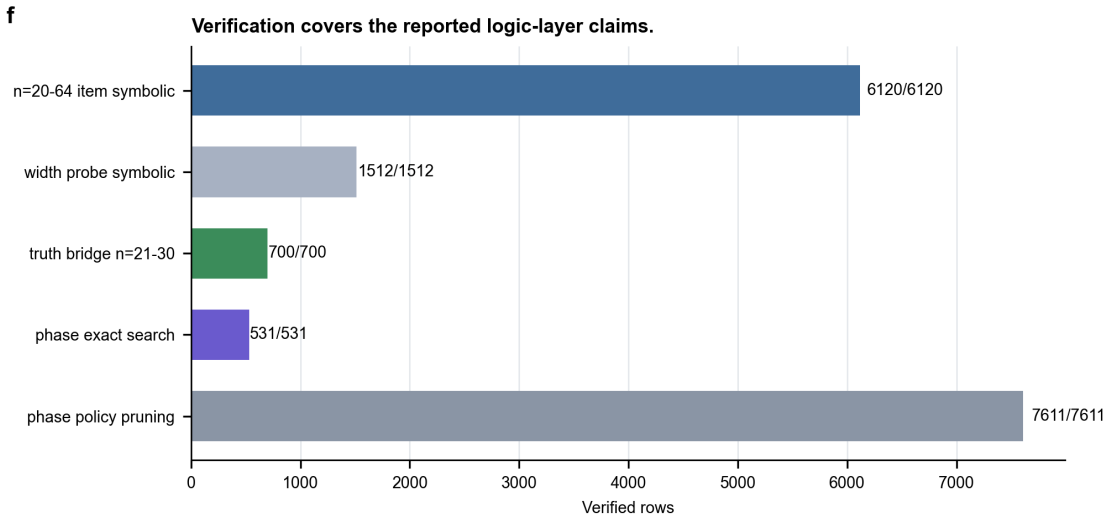

Figure 7: **Verification coverage.** Large term-set experiments are validated symbolically; phase-search and phase-policy rows are checked up to global phase; $n = 21$–26 bridge slices add complete truth table checking on selected generated functions.

Table 61: Scaling and resource audit for high-dimensional experiments. The representative resource mean is method-specific; it is not an average over all baselines in the slice.

Slice	n	Fn./settings	Rows verified	Representative resource mean	Boundary
Large term-set frontier	$24, 28, 32, 40$	96	960/960	score 1090.0; T/CNOT/depth 986/1593/1593; anc. 5.3; time 7.40s	Term-set validation; no full truth table beyond bridge slices.
Stage-gated frontier	$24, 28, 32, 40$	96	96/96	score 1089.7; T/CNOT/depth 985/1593/1593; anc. 5.3; time 10.57s	Controller audit; still term-set symbolic verification.
Action-width probe	$20, 28, 40$	216	1512/1512	score 1130.4; T/CNOT/depth 1023/1648/1648; anc. 5.2; time 1.27s	Sensitivity check; wider root screens are not claimed as better.
Ultra-scale term-set stress	$48, 56, 64$	48	480/480	score 1182.6; T/CNOT/depth 1070/1728/1728; anc. 5.4; time 17.11s	Ultra-scale symbolic stress; no truth-table enumeration.
Complete truth-table bridge	21–30	70	700/700	score 1105.4; T/CNOT/depth 1000/1616/1616; anc. 5.3; time 4.68s; truth check 2.34s	Complete checking on generated bridge slices only.

Table 62: Ultra-scale $n = 48, 56, 64$ term-set stress test. Rows are verified by symbolic plan expansion and emitted-circuit ANF simulation. The learned frontier is reported as a budget controller: it saves time against the full measured frontier but accepts a small score gap on this harder slice.

Scope	Target	Baseline	W/L/T	Δ score	Δ time
$n = 48, 56, 64$	depth-4 screen	depth-2 screen	24/0/24	-1.81%	+488.77%
$n = 48, 56, 64$	all-depth frontier	depth-2 screen	24/0/24	-1.81%	+809.95%
$n = 48, 56, 64$	learned frontier	depth-2 screen	10/0/38	-0.84%	+244.67%
$n = 48, 56, 64$	learned frontier	depth-4 screen	0/14/34	+1.02%	-35.99%
$n = 48, 56, 64$	learned frontier	all-depth frontier	0/14/34	+1.02%	-61.61%
$n = 48$	learned frontier	all-depth frontier	0/1/15	+0.07%	-53.13%
$n = 56$	learned frontier	all-depth frontier	0/6/10	+1.14%	-60.06%
$n = 64$	learned frontier	all-depth frontier	0/7/9	+1.85%	-71.64%

Table 63 separates the same stress slice into raw logical resources. The learned frontier is not the all-depth quality ceiling, but it lowers mean score, T-count, CNOT count, depth, and T-depth proxy relative to the cheap depth-2 screen while reducing planning time relative to the full measured frontier.

Table 63: Ultra-scale $n = 48, 56, 64$ resource profile. Means are over the 48 verified generated term-set functions in the stress slice; lower is better.

Method	Score	T	CNOT	Depth	Anc.	T-depth	Anc.-life	Time(s)
Direct AND	1378.1	1250.4	2002.3	2002.4	5.02	595.8	0.0	0.00
Depth-2 screen	1202.8	1088.8	1755.6	1755.7	5.38	517.8	23.2	2.80
Depth-4 screen	1168.9	1057.6	1709.4	1709.4	5.40	503.1	29.7	27.53
All-depth frontier	1168.9	1057.6	1709.4	1709.4	5.40	503.1	29.7	41.27
Learned frontier	1182.6	1070.2	1727.7	1727.8	5.38	509.1	26.5	17.11

Table 64: Verification scope and boundary.

Check	Current coverage	Boundary
$n = 20$–64 item-level symbolic runs	6120/6120	Not full truth-table enumeration.
Width-probe symbolic runs	1512/1512	Supports the default action width, but remains term-set validation.
$n = 21$–30 complete bridge	700/700	Small generated slices; not exhaustive coverage of all high-dimensional functions.
Exact phase-search rows	531/531	Verified up to global phase; no rotation-sequence output.
Phase-policy pruning rows	7611/7611	Learned shortlist rows verified up to global phase; still no rotation-sequence output.
Rotation-sequence smoke angles	20/20	Source-derived single-qubit Clifford+T strings at coarse tolerance; not high-precision gridsynth.

7 Discussion

The evidence supports a specific, bounded claim: Resource-NMCTS is a logical-layer resource-constrained Boolean oracle synthesis method that improves T-count and weighted score over several fixed-representation and external-toolchain baselines. The claim is not that Resource-NMCTS dominates every metric. SSHR remains a strong CNOT-oriented small-function baseline;

CirKit AIG/MC remains a strong depth-oriented external probe; and RevKit CLI uses fewer auxiliary lines on most traditional functions.

The ablations also calibrate the role of AI. The neural prior is useful, but the largest gain comes from the searchable algebraic action space and from guarded portfolio selection. This is a better fit for the evidence than claiming that deep reinforcement learning alone drives the result. The Boolean-ring structural audit further separates resource quality from planning cost: at $n = 14$ and $n = 16$ the guard improves score by 0.34–0.82% over the pairwise-wide baseline; at $n = 20$ a deeper screen gives a 7.47% score reduction over the old Resource baseline with an ancilla tradeoff; and the $n = 18$ screen-only row is much faster but worse in score. The sparse depth-frontier audit suggests a simpler planning-cost reduction inside the current frontier: depth 3 was never needed once depth 4 was measured in the audited slices, so a depth-2/4 frontier exactly matched all-depth resources while removing roughly one quarter of frontier evaluation time. This is an empirical controller for the present generated slices, not a proof that depth 3 is universally dominated. The learned sparse depth-4 gate adds a narrower AI-control result: after the sparse frontier has removed depth 3, the gate skips some depth-4 evaluations without changing score in either the held-out split or the independent multi-seed audit under its conservative threshold. The large, cost-aware, sparse-gated, and staged frontier controllers should be read as operating points on the same search frontier: quality-oriented single-shot selection, faster cost-aware selection, learned sparse-budget control, and a validation-calibrated staged teacher. In the phase/Rz branch, affine and polarity search give verified improvements, and the rank-trained diverse shortlist now tracks the wide-search frontier with 6.25% of the exact evaluations. However, random-repeat margins remain small, so this branch should be read as learned pruning evidence rather than as a solved rotation-synthesis policy. The next target is rotation-spectrum-aware optimization and sequence-level audit.

Table 65 collects the main threats to validity and the audit evidence that mitigates them. It is included to make the limitation boundary explicit before the final conclusion: the paper is a logical-layer resource study with layered baselines, bounded learned-control claims, symbolic large-instance verification, and reproducible packaging, not a hardware-mapped compiler or universal optimality result.

Table 65: Threats-to-validity and mitigation audit. Each row names a reviewer risk, points to the generated evidence that mitigates it, and preserves the residual boundary that the current experiments do not cross.

Threat	Risk	Mitigation evidence	Residual boundary
Logical-layer abstraction	Readers may infer hardware routing, native-gate, noise, or magic-state-factory resource claims.	The abstract, discussion, claim-scope lint, and payload README state that the paper reports logical-layer resources only.	No placement, routing, native-gate scheduling, noise, or factory-level accounting is evaluated.
Weighted-score aggregation	A single score could hide unfavorable CNOT, depth, ancilla, or lifetime behavior.	Raw dominance, schedule-proxy, weight-robustness, resource-weight sensitivity, and counterpoint audits expose resource dimensions separately from the weighted score.	The method is presented as a strong T-count and weighted-score point, not complete raw-resource dominance.
Baseline comparability	External logic-network, reversible, phase, and ROS-style rows could be mistaken for one fully uniform compiler benchmark.	The comparison matrices assign each baseline family a role, verified evidence, usable claim, and excluded claim.	ROS-style and logic-network probes remain logical proxies; RevKit rows are exact-oracle or phase proxies, not mapped circuits.
AI/MCTS attribution	The manuscript could overstate deep learning or MCTS as the sole reason for the gains.	Search-control, random-prior, random-depth, and learned-control audits separate algebraic search space, MCTS, Pareto archives, and learned ranking.	Learned controls are bounded and sometimes runtime-negative; they are not promoted as the whole resource-reduction mechanism.
High-dimensional verification envelope	Large-n symbolic rows may be read as exhaustive truth-table benchmarking or optimality evidence.	Scaling, ultra-scale, resource-profile, and truth-bridge audits separate symbolic verification from complete truth-table checks.	Complete truth-table verification is limited to generated $n = 21$–30 bridge slices; $n = 31$–64 rows are not exhaustive truth-table enumerations.
Statistical and benchmark representativeness	Mean improvements could be driven by outliers or by an unrepresentative benchmark subset.	Paired statistics, bootstrap intervals, and headline consistency checks recompute matched comparisons from CSV evidence.	The evidence covers the generated and benchmarked slices in the artifact package, not all Boolean functions.
Reproducibility and package drift	A complete-looking manuscript could drift from the scripts, raw rows, tables, and archive manifest.	Reproducibility, artifact-registry, and archive-manifest audits connect scripts, raw rows, summaries, tables, and manuscript payload groups.	The lightweight rebuild regenerates paper-facing artifacts but does not rerun heavy raw sweeps or neural training jobs.

Several limitations are deliberate. All results are logical-layer estimates; the paper does not model routing, hardware connectivity, native gate sets, noise, or magic-state factories. The ROS-style LUT result is a proxy rather than a full ROS reproduction with SAT garbage management; the line-aware LUT reselection only tests parameter sensitivity within the same proxy. The Caterpillar source-family audit and XAG API probe add a verified implementation-family counterpoint, but they are still not the official ROS flow. The RevKit CLI baseline is a real exact-oracle reversible-synthesis probe, but CNOT/depth are derived from RevKit's Toffoli-control distribution and the experiment is still not a hardware mapping. High-dimensional symbolic checks validate emitted circuit semantics but do not replace full truth-table enumeration beyond the bridge slices.

8 Conclusion

We presented Resource-NMCTS, a neural Monte Carlo tree search framework for resource-constrained quantum Boolean oracle synthesis over verifiable ANF/FPRM actions. On matched small-function benchmarks, the method substantially lowers T-count and weighted score relative to direct ANF, ESOP, SSHR-H, and ESOP-MILP baselines. On external probes based on ROS-style LUT mapping, mockturtle, Caterpillar API, CirKit, and legacy RevKit CLI reversible

synthesis, the method retains a strong weighted-score advantage while exposing clear CNOT, depth, and ancilla tradeoffs. Large symbolic experiments and $n = 21$–30 complete truth-table bridges support the semantic correctness of the logical-layer circuits. The sparse depth-frontier analysis further reduces high-dimensional planning cost by skipping depth 3 in the audited frontier, and the learned sparse depth-4 gate preserves sparse-frontier score across both held-out and independent-seed audits while removing additional depth-4 evaluations. The staged frontier analysis remains a validation-calibrated controller when depth-4 evaluation itself should be conditional. The next step is not to claim hardware-level optimality, but to strengthen the phase/Rz policy and complete fuller toolchain reproductions while keeping the claim at the logical synthesis layer.

Data and Code Availability

The artifact package used for this manuscript is maintained in the project repository under `resource_nmcts_experiment/`. The directory contains the synthesis code, neural and search-policy scripts, external-toolchain probe drivers, generated raw and summary CSV files, manifest JSON files, LaTeX table fragments, figure source data, and the compiled manuscript PDF. The main reproducibility entry points are the `run_*.py`, `train_*.py`, and `analyze_*.py` scripts listed in the project README; generated measurements are stored in `results/raw_*.csv`, `results/summary_*.csv`, and `results/manifest_*.json`. The paper tables are generated under `paper_latex/tables/`, and the submission figures and source data are stored under `paper_latex/figures/submission_v36/`. The lightweight derived submission package can be rebuilt with `./rebuild_submission_package.sh`; this regenerates paper-facing tables, figures, the submission archive manifest, the uploadable payload archive, audits, and the English PDF from existing experiment artifacts, but does not rerun raw benchmark sweeps, external-toolchain probes, or neural training jobs. The generated payload archive `submission_package/dist/resource_nmcts_submission_payload.tar.gz` is accompanied by a SHA256 sidecar and a JSON file manifest. The archive manifest records category-level hashes for stable payload groups; terminal submission audits and the compiled PDF are checked separately to avoid self-referential digests. The experiments use the `mcts-qoracle` Conda environment and the direct interpreter `/opt/anaconda3/envs/mcts-qoracle/bin/python`. External-tool rows are reported with manifest-level provenance; hardware mapping, routing, and magic-state-factory artifacts are intentionally absent because they are outside the logical-layer scope of the study.

References

[1] Robert K. Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010.

[2] Ayushi Dubal, David Kremer, Simon Martiel, Victor Villar, Derek Wang, and Juan Cruz-Benito. Pauli network circuit synthesis with reinforcement learning, 2025. arXiv:2503.14448.

[3] Florian Fürrutter, Gorka Muñoz-Gil, and Hans J. Briegel. Quantum circuit synthesis with diffusion models. *Nature Machine Intelligence*, 6:515–524, 2024.

[4] David Kremer, Ali Javadi-Abhari, and Priyanka Mukhopadhyay. Optimizing the non-Clifford-count in unitary synthesis using reinforcement learning, 2025. arXiv:2509.21709.

[5] Mulundano Machiya, Matt Menickelly, Paul Hovland, and Ji Liu. MonteQ: A monte carlo tree search based quantum circuit synthesis framework, 2026. arXiv:2604.19029.

[6] Giulia Meuli. The single-target gate benchmark suite. GitHub repository, 2020. Compilation results for 4- and 5-input Boolean-function spectral representatives; accessed 2026-07-09.

[7] Giulia Meuli and collaborators. Caterpillar: Quantum circuit synthesis for boolean functions. https://github.com/gmeuli/caterpillar, 2026. C++17 library used as a local source/API probe for Boolean-function quantum-circuit synthesis and quantum memory-management components.

[8] Giulia Meuli, Mathias Soeken, Earl Campbell, Martin Roetteler, and Giovanni De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2019.

[9] Giulia Meuli, Mathias Soeken, and Giovanni De Micheli. Xor-and-inverter graphs for quantum compilation. *npj Quantum Information*, 8(1):7, 2022.

[10] Giulia Meuli, Mathias Soeken, Martin Roetteler, and Giovanni De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.

[11] Sebastian Rietsch, Abhishek Y. Dubey, Christian Ufrecht, Maniraman Periyasamy, Axel Plinge, Christopher Mutschler, and Daniel D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 824–835. IEEE, 2024.

[12] Jordi Riu, Jan Nogué, Gerard Vilaplana, Artur Garcia-Saez, and Marta P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.

[13] Neil J. Ross and Peter Selinger. Optimal ancilla-free Clifford+T approximation of Z-rotations. *Quantum Information & Computation*, 16(11–12):901–953, 2016.

[14] Mathias Soeken and collaborators. CirKit: Logic synthesis and optimization framework. https://github.com/msoeken/cirkit, 2026. CirKit 3 shell and legacy RevKit/CirKit command-line source.

[15] Mathias Soeken and collaborators. mockturtle: Logic network optimization and synthesis. https://mockturtle.readthedocs.io/, 2026. Official-header KLUT-to-XAG probe source.

[16] Mathias Soeken and collaborators. RevKit: Reversible logic and quantum circuit compilation. https://github.com/msoeken/revkit, 2026. Accessed as a local Python API and legacy command-line reversible-synthesis toolchain.

[17] Mathias Soeken, Stefan Frehse, Robert Wille, and Rolf Drechsler. RevKit: An open source toolkit for the design of reversible circuits. In *Reversible Computation*, volume 7165 of *Lecture Notes in Computer Science*, pages 64–76. Springer, 2012.

[18] Mathias Soeken, Heinz Riener, Winston Haaswijk, Eleonora Testa, Bruno Schmitt, Giulia Meuli, Fereshte Mozafari, Siang-Yun Lee, Alessandro Tempia Calvino, Dewmini Sudara Marakkalage, and Giovanni De Micheli. The EPFL logic synthesis libraries, 2018. arXiv:1805.05121.

[19] Lukas Theißinger, Thore Gerlach, David Berghaus, and Christian Bauckhage. Beyond reinforcement learning: Fast and scalable quantum circuit synthesis, 2026. arXiv:2602.15146.

[20] Dimitrios Tsaras, Antoine Grosnit, Lei Chen, Zhiyao Xie, Haitham Bou-Ammar, and Mingxuan Yuan. ShortCircuit: AlphaZero-driven circuit design, 2024. arXiv:2408.09858.

[21] Xavier Valcarce, Bastien Grivet, and Nicolas Sangouard. Unitary synthesis with AlphaZero via dynamic circuits, 2025. arXiv:2508.21217.

[22] Pei-Yong Wang, Muhammad Usman, Udaya Parampalli, Lloyd C. L. Hollenberg, and Casey R. Myers. Automated quantum circuit design with nested monte carlo tree search. *IEEE Transactions on Quantum Engineering*, 4:1–20, 2023.

[23] Mathias Weiden, Ed Younis, Justin Kalloor, John Kubiatowicz, and Costin Iancu. Improving quantum circuit synthesis with machine learning. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2023.

[24] Robert Wille and Rolf Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of the 46th Annual Design Automation Conference*, DAC '09, pages 270–275. ACM, 2009.

[25] Mingfei Yu, Alessandro Tempia Calvino, Mathias Soeken, and Giovanni De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *2025 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 930–937. ACM, 2025.

[26] Qiang Zheng, Yongzhen Xu, Jiaxi Zhang, Zhaofeng Su, and Shenggen Zheng. CNOT oriented synthesis for small-scale boolean functions using spatial structures of parallelotopes. In *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2025.

[27] Xiangzhen Zhou, Yuan Feng, and Sanjiang Li. Quantum circuit transformation: A monte carlo tree search framework. *ACM Transactions on Design Automation of Electronic Systems*, 27(6):1–27, 2022.